

UIROBOT

User Manual

UIM342 / UIM342S / UIM342XS
Motion Controller for Servo Stepper Motor
with CAN Interface



UIM342 / UIM342S / UIM342XS

Intellectual Property Protection Statement

- UIROBOT products meet the specifications contained in their UIROBOT user manual.
- UIROBOT is willing to work with customers who is concerned about intellectual property protection.
- Any attempt to destroy the code protection function of UIROBOT products may be regarded as a violation of intellectual property protection laws and regulations. If this behavior leads to the acquisition of software or other results protected by intellectual property rights without UIROBOT's authorization, UIROBOT has the right to file a lawsuit to stop this behavior in accordance with the Act.

Disclaimer

Information contained in this publication regarding device applications and the like is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. UIROBOT MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION, INCLUDING BUT NOT LIMITED TO ITS CONDITION, QUALITY, PERFORMANCE, MERCHANTABILITY OR FITNESS FOR PURPOSE. UIROBOT disclaims all liability arising from this information and its use. Use of UIROBOT Products in life support and/or safety applications is entirely at the buyer's risk, and the buyer agrees to defend, indemnify and hold harmless UIROBOT from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any UIROBOT intellectual property rights.

Trademark and Design Statement

- UIROBOT's name and logo is a registered trademark of UIROBOT in China and other countries or regions.
- The appearance designs of UIROBOT's products have applied for patent protection.

Contact information

United Intelligence Robotics (Zhejiang) Co., Ltd.

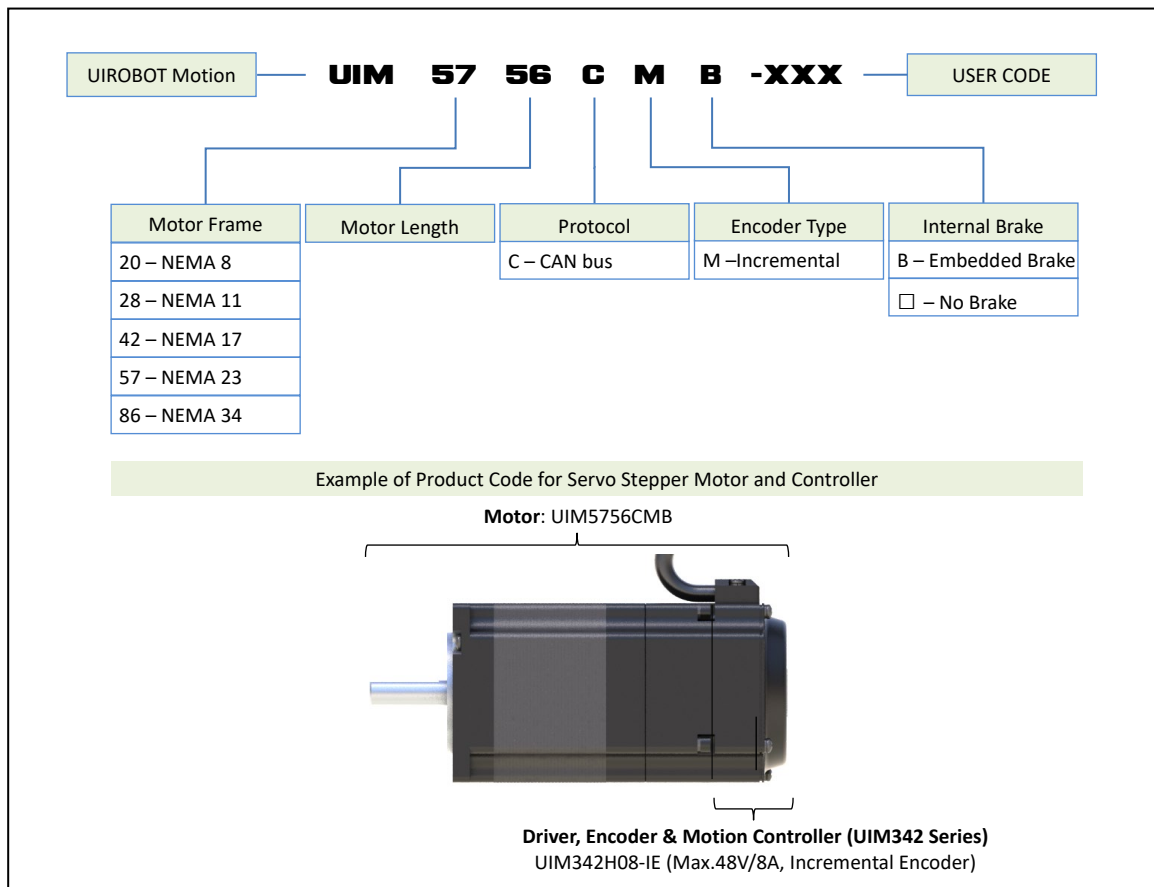
Address: Bldg. B3, Innovation Park, Pinghu, Jiaxing City, Zhejiang Province, China 314200

Tel: +86 0573-85235952

Email: support@uirobot-zj.com

Website: www.uirobot.com

UIM342 Servo Stepper Motor Order Code



Motion Controller with CAN Interface

UIM342 / UIM342S / UIM342XS Motion Controller for Servo Stepper Motor With CAN Bus Interface

The UIM342 series are advanced motion controllers designed for servo stepper motors, featuring a CAN Bus interface.

These controllers integrate several key functional modules: *Communication Unit / Motion Control Unit / Motor Driver / Feedback Control Unit / Input Logic Control and DSP System* (for processing instructions, replying, notifications, coordinating other functional modules, executing preloaded user programs, and providing real-time status updates)

The major control loop is executed within 1 millisecond, ensuring responsiveness.

Key Features:

- **Absolute Multi-Turn Encoder, with internal rechargeable battery.**
- **High-Performance DSP System:** Executes all control loops within 1 millisecond.
- **Comprehensive SDK:** Includes SDK, DLL, LIB, and SO files.
- **Sample Codes:** Available in C++, C#, and other languages.
- **Cross-Platform Support:** Compatible with Linux, Windows 32-bit, and 64-bit systems.

Control System

- Robust DSP hardware
- Fault tolerance, fail safe user interface
- Input and event change notification

Advanced Motion

- Interpolated motion for multi-axis coordination, speed control and position control
- High positioning accuracy, high response, Maximum speed up to 3000 rpm, depending on the motor type
- Backlash compensation
- Stall detection

Motor Driver

- Wide voltage input: 24 ~ 48 VDC⁽¹⁾
- Adjustable phase current: up to 8 A^{·(1)}
- Micro-stepping resolution: 1 ~ 1/128
- No vibration or noise at low speeds.
- Protection: overcurrent, overvoltage, overheating, etc.

I/O Logic Control

- 3 Digital Inputs, 1 Output (5V / 50mA)
- Input change notification
- 13 programmable actions can be attached to the I/O trigger

CAN Networking

- Active CAN 2.0, 1 Mbps Max.
- 3 types of IDs (Node ID, Group ID, Global ID) for synchronized motion.

Other features

- Emergency lockdown
- Integrated design with motor
- Aluminum alloy casing, sturdy and durable for easy heat dissipation
- Size as small as 20 mm x 20 mm x 16 mm

Note:

(1) Depending on the actual controller.

CONTENTS

1.0	Introduction	15
1.1	Communication	16
1.2	Motion Control	16
1.3	Motor Driver	16
1.4	Input Logic Control	16
1.5	Logic after Motor Stall	17
1.6	Real Time Notification	17
1.7	System development SDK	17
1.8	Connect to User Devices	18
2.0	Protocol.....	19
2.1	Instruction and Reply (ACK).....	19
2.2	Mnemonic.....	20
2.3	Error Report	20
2.4	Real-Time Status and Alarm Notification	21
2.5	Direct CAN Communication	22
2.6	RS232 / Ethernet Gateway.....	24
3.0	Motion Control.....	25
3.1	Direction of Rotation.....	25
3.2	Motion Control Modes	25
3.3	Acceleration, Deceleration and Cut-in speed	26
3.4	Backlash Compensation	27
3.5	Closed Loop / Open Loop Control.....	27
3.6	Pulse Rate vs Speed.....	28
3.7	Stall Detection	28
3.8	Acquiring Motion Status	29
3.9	Internally Controlled Brake	30
3.10	Software Limits.....	30
3.11	Built-in Functional Logics	31
4.0	I/O & Input Logic Control.....	34
4.1	Wiring the Sensor.....	35
4.2	Trigger type and Input Filter	35
4.3	Configure Input Logic Action	36
4.4	Real-time Notification for Input Change	37
5.0	Instruction Set.....	38
6.1	PP[i] Protocol Parameter	39
6.2	IC[i] Initial Configuration.....	41
6.3	IE[i] Inform Enable	45
6.4	ML Model	46
6.5	SN Serial Number.....	47
6.6	ER[i] Error Report	48
6.7	QE[i] Quadrature Encoder.....	49
6.8	SY[i] System Operation	51
6.9	MT[i] Motor Driver	52
6.10	MO Motor Driver On /Off	55
6.11	BG Begin Motion.....	56
6.12	ST Stops Motion.....	57
6.13	MF Motion Parameter Frame.....	58
6.14	AC Acceleration	59
6.15	DC Deceleration	60
6.16	SS Cut-in Speed.....	61
6.17	SD Stop Deceleration	62

Motion Controller with CAN Interface

6.18	JV Jog Velocity.....	63
6.19	SP PTP Speed.....	64
6.20	PR Position Relative.....	65
6.21	PA Position Absolute.....	66
6.22	OG Set Origin	67
6.23	BL Backlash Compensation.....	68
6.24	LM[i] Software Limits	69
6.25	MS[i] Motion Status	70
6.26	DV[i] Desired Values	71
6.27	IL[i] Input Logic.....	72
6.28	TG[i] Trigger.....	73
6.29	DI Digital I/O	74
6.30	RT Real-Time Inform.....	75
6.31	TI Temperature Indication.....	76
Appendix-1 RTU CRC16 Source Code.....		77
Appendix-2 LED Indicator Patterns and Meanings.....		78

SAFETY

To prevent personal injury and property damage, please pay attention to the following before use:

	Precautions	Consequences of Neglect
	Do not use it in humid , corrosive, flammable gas environments or places near flammable substances	Fire / Malfunction
	Do not use the wire when it is soaked in oil/water	Fire / Malfunction
	For motors with shaft keyway, do not touch the keyway with bare hands	Personal injury
	Never touch the rotating parts of a running motor	Personal injury
	Do not touch the running motor, it may be very hot	Personal injury
	Do not cause the wires to be damaged or subjected to excessive external force, heavy pressure, or clamping	Fire / Malfunction
	Do not hold the cable or motor shaft when transporting	Personal injury / Malfunction
	Never hit the motor	Malfunction
	Do not frequently power on / off	Malfunction
	Never modify, disassemble or repair by yourself	Fire / Malfunction / Personal injury
	Power supply voltage must meet the product requirements, and the Power supply current must be 1.5 times larger than the product requirements	Malfunction
	Cut off the power when not in use for a long time	Fire / Malfunction / Personal injury

MAINTENANCE

Please perform regular maintenance and inspection on the controller for safe use. Please pay attention to the following during maintenance and inspection:

1. When performing the insulation test on the drive, be sure to disconnect all connections.
2. Do not use gasoline, thinner, alcohol, acidic and alkaline cleaning agents to avoid damage to the casing.

Daily inspections and periodic inspections should be carried out according to the following items.

Type	Period	Check Item
Daily Inspection	Every day	• Confirm the operating temperature and humidity • No foreign matter entering • Abnormal vibration, sound and odor • Abnormal power supply voltage • Damaged wiring parts
Periodic Inspection	1 year	• No looseness in the fastening parts • Broken terminal blocks and loose fastening parts

HARDWARE

Figure 0-1: UIM342XS Wiring Diagram

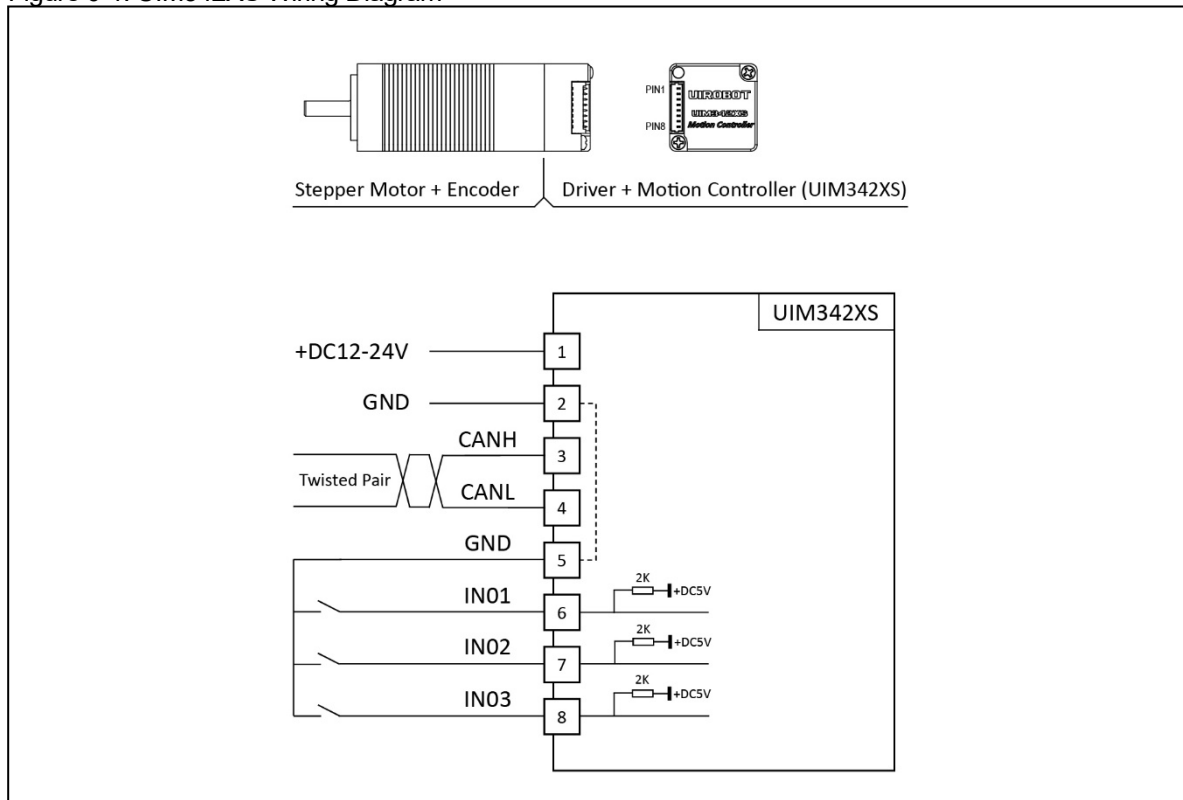
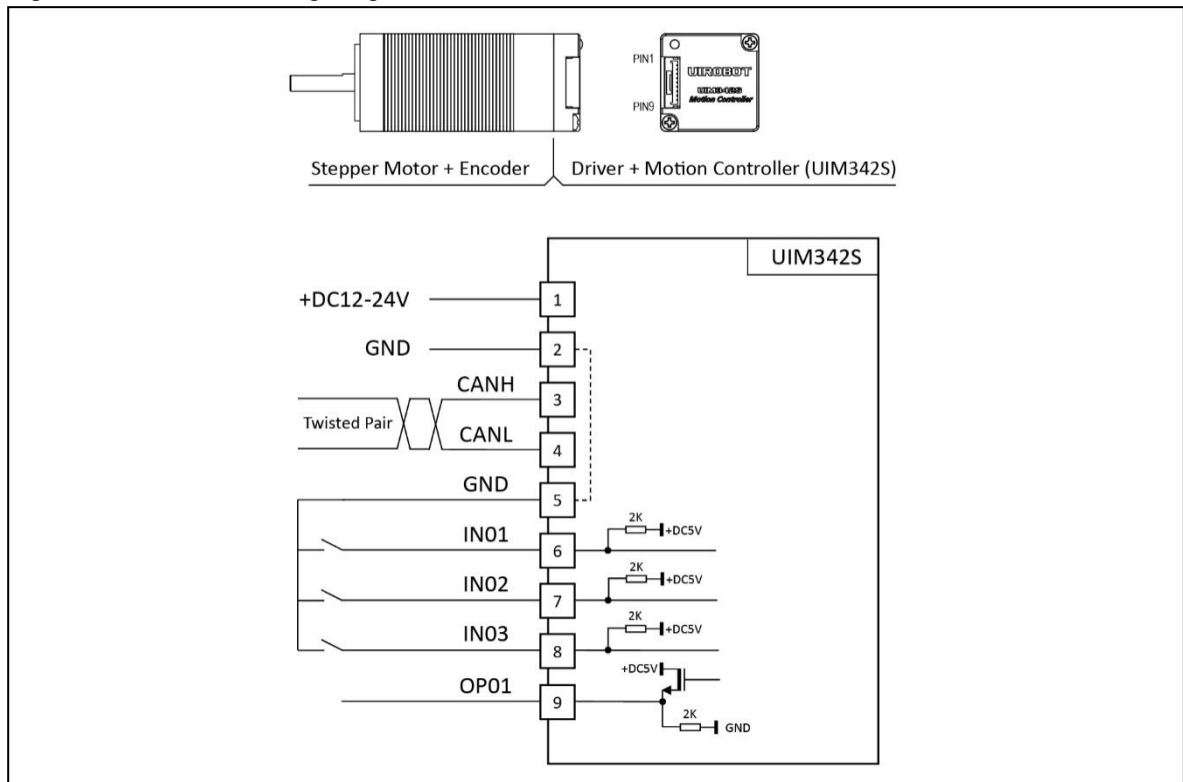


Figure 0-2: UIM342S Wiring Diagram



UIM342 / UIM342S / UIM342XS

Figure 0-3: UIM342C Wiring Diagram (Socket Style)

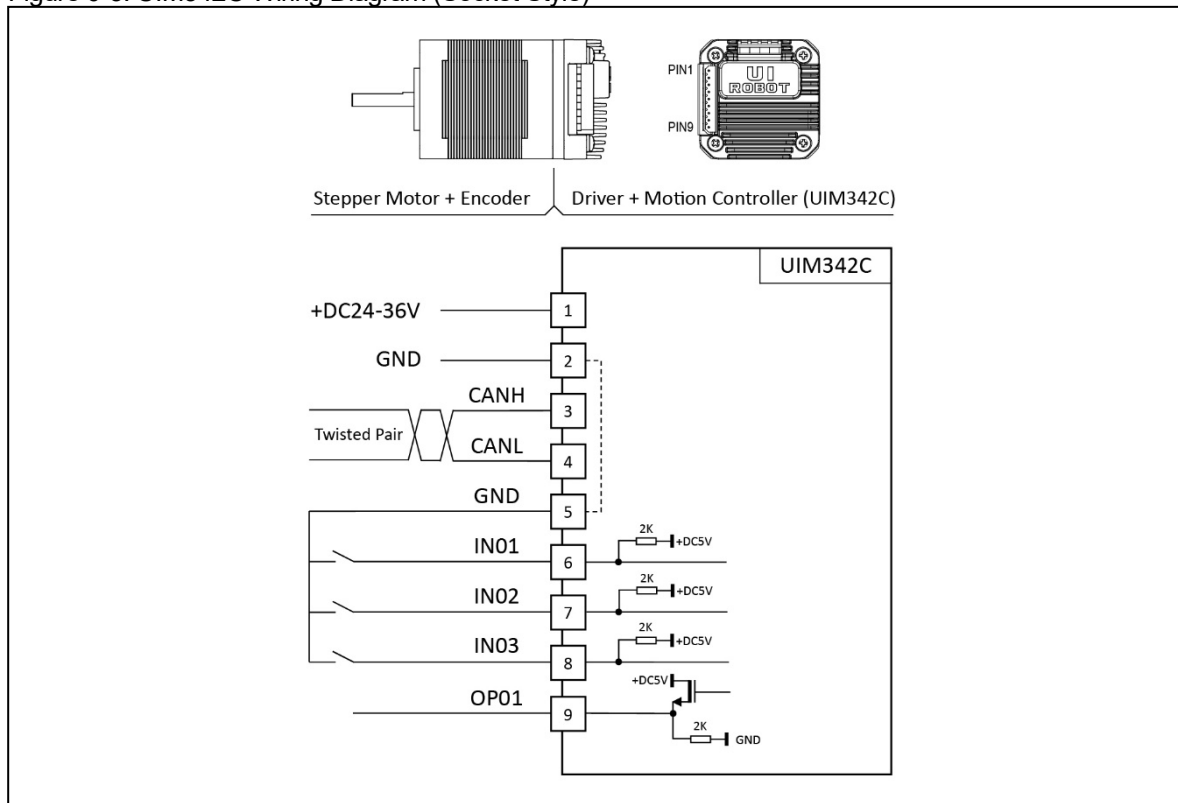
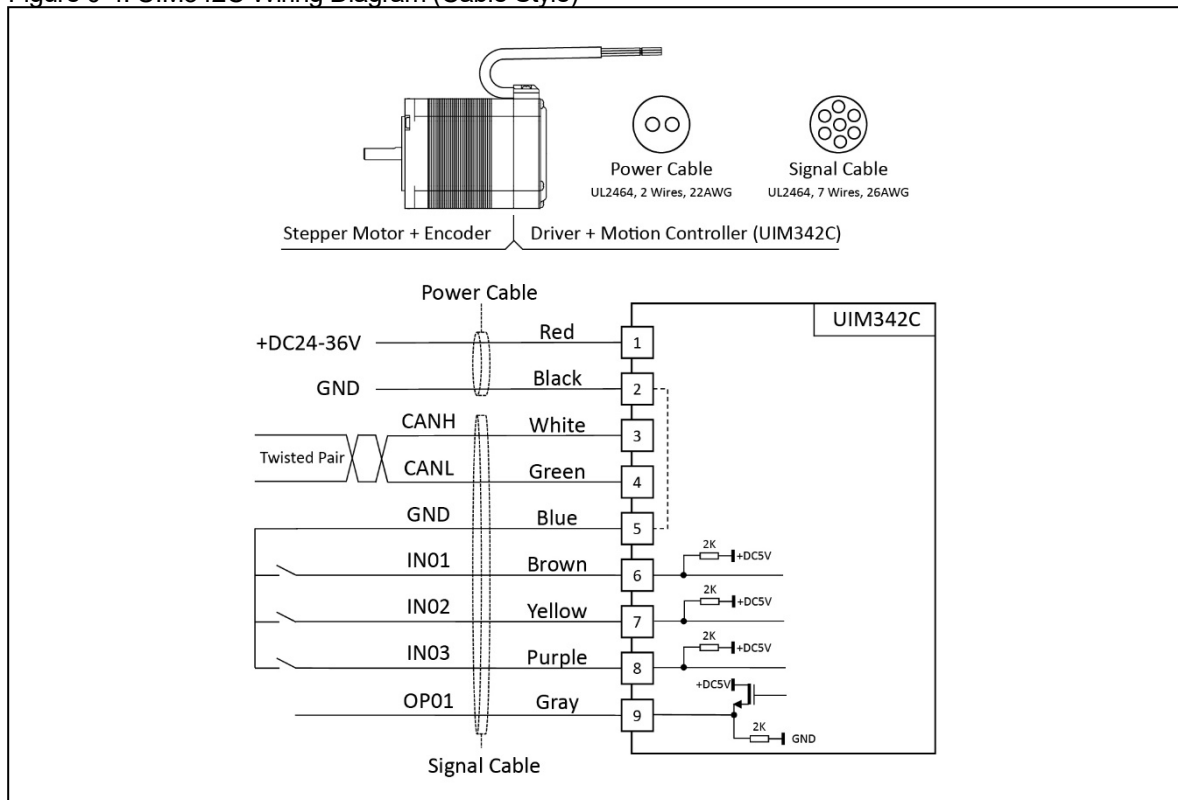


Figure 0-4: UIM342C Wiring Diagram (Cable Style)



INSTRUCTION SUMMARY

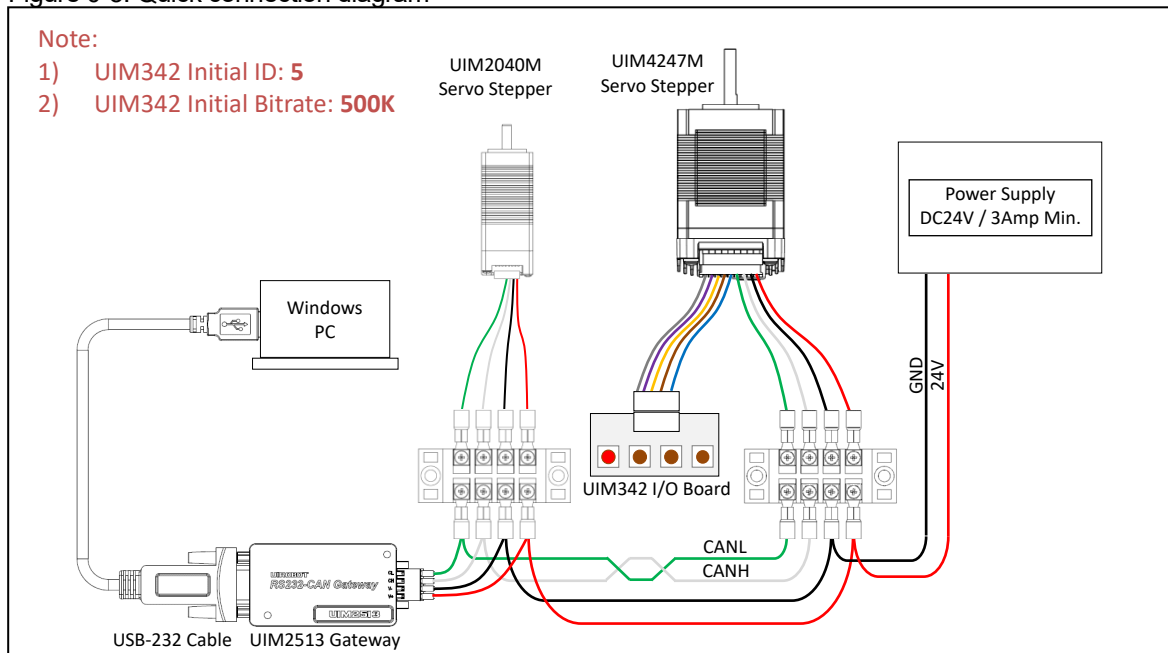
Classification	Mnemonic	Control Word	Function	Object	Chapter
Protocol	PP[i]	0x01	Set / Get	Protocol Parameters	6.1
System	IC[i]	0x06	Set / Get	Initial Configuration	6.2
	IE[i]	0x07	Set / Get	Information Enable	6.3
	ML	0x0B	Get	Model	6.4
	SN	0x0C	Get	Serial Number	6.5
	ER[i]	0x0F	Clear / Get	Error Report	6.6
	QE[i]	0x3D	Set / Get	Quadrature Encoder	6.7
	SY[i]	0x7E	Set	System Operation	6.8
Motor Driver	MT[i]	0x10	Set / Get	Motor Driver parameters	6.9
	MO	0x15	Set / Get	Motor Driver On /Off	6.10
Motion Control	BG	0x16	Set	Begin Motion	6.11
	ST	0x17	Set	Stop Motion	6.12
	MF	0x18	Get	Motion Parameter Frame	6.13
	AC	0x19	Set / Get	Acceleration	6.14
	DC	0x1A	Set / Get	Deceleration	6.15
	SS	0x1B	Set / Get	Cut-in speed	6.16
	SD	0x1C	Set / Get	Stop Deceleration	6.17
	JV	0x1D	Set / Get	Jog Velocity	6.18
	SP	0x1E	Set / Get	PTP Speed	6.19
	PR	0x1F	Set / Get	Position Relative	6.20
	PA	0x20	Set / Get	Position Absolute	6.21
	OG	0x21	Set	Set Origin	6.22
	LM[i]	0x2C	Set / Get	Software Limits	6.23
	BL	0x2D	Set / Get	Backlash Compensation	6.24
	MS[i]	0x11	Clear / Get	Motion Status	6.25
	DV[i]	0x2E	Get	Desired Values	6.26
I / O	IL[i]	0x34	Set / Get	Input Logic	6.27
	TG[i]	0x35	Set / Get	Trigger	6.28
	DI	0x37	Set / Get	Digital I/O	6.29
Notification	RT	0x5A	Auto Sent	Real-Time Inform	6.30

QUICK START

The following explains how to quickly build and run a motor system consisting of 2 servo stepper motors equipped with UIM342 controllers and 1 UIM2513 gateway. *For the convenience of users, the development kit (UIMEVA_342KIT) provided by UIROBOT Company includes the experimental materials used in the system (excluding Windows PC).*

- 1) Connect the wiring as shown below. Be sure to check the wiring again before powering on to make sure it is correct.

Figure 0-5: Quick connection diagram

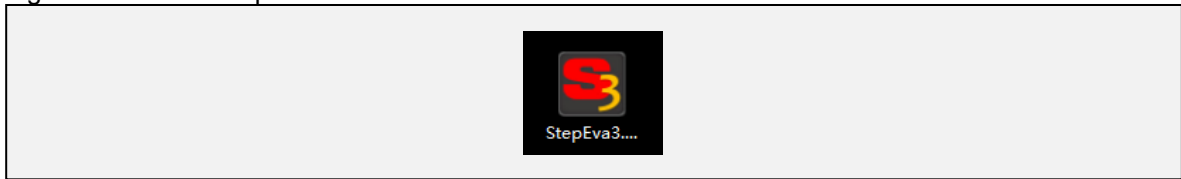


WARNING

- Avoid using star connections. When the CAN cable length exceeds 20 meters, dedicated CAN cables should be used. The length of branch cable to each node should not exceed 20 cm.
 - UIM2513 has a built-in terminal resistor, which can be activated by the toggle switch. It is recommended to connect a 120-ohm terminal resistor to the other end of the CAN cable.
 - **Strictly prohibit Hot-Plugging while the power is on.** Hot-plugging may lead to ground loss (i.e. the power supply V+ is connected while V- is disconnected). In such instances, power V+ will flow into other UIM devices via the CAN cable, causing the burnout of multiple UIM devices.
 - Connect all UIM devices to a common ground. Activating a high-power device can raise the voltage on one ground significantly. Without a common ground, this elevated voltage may flow via the CAN cable to the ground of other UIM devices, risking the burnout of multiple devices.
- 2) Download and click to run the Windows based control panel “StepEva3”.

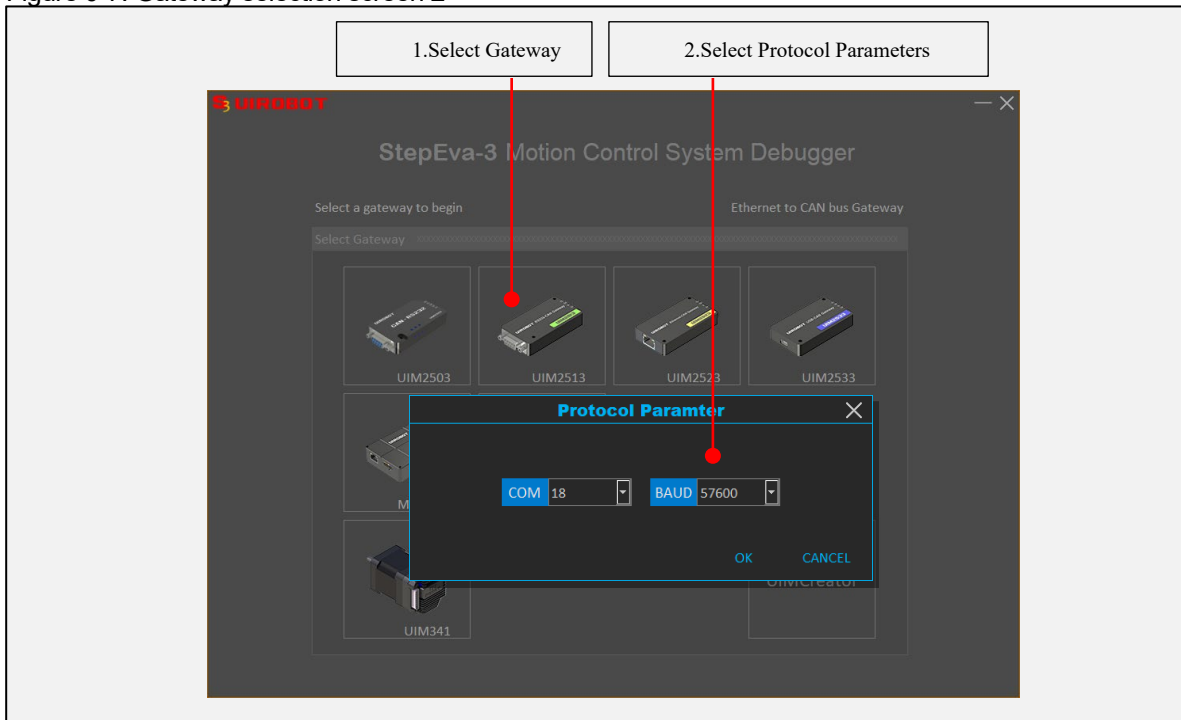
Motion Controller with CAN Interface

Figure 0-6: Icon of StepEva3



3) UIM2513 (RS232 -CAN) should be selected here.

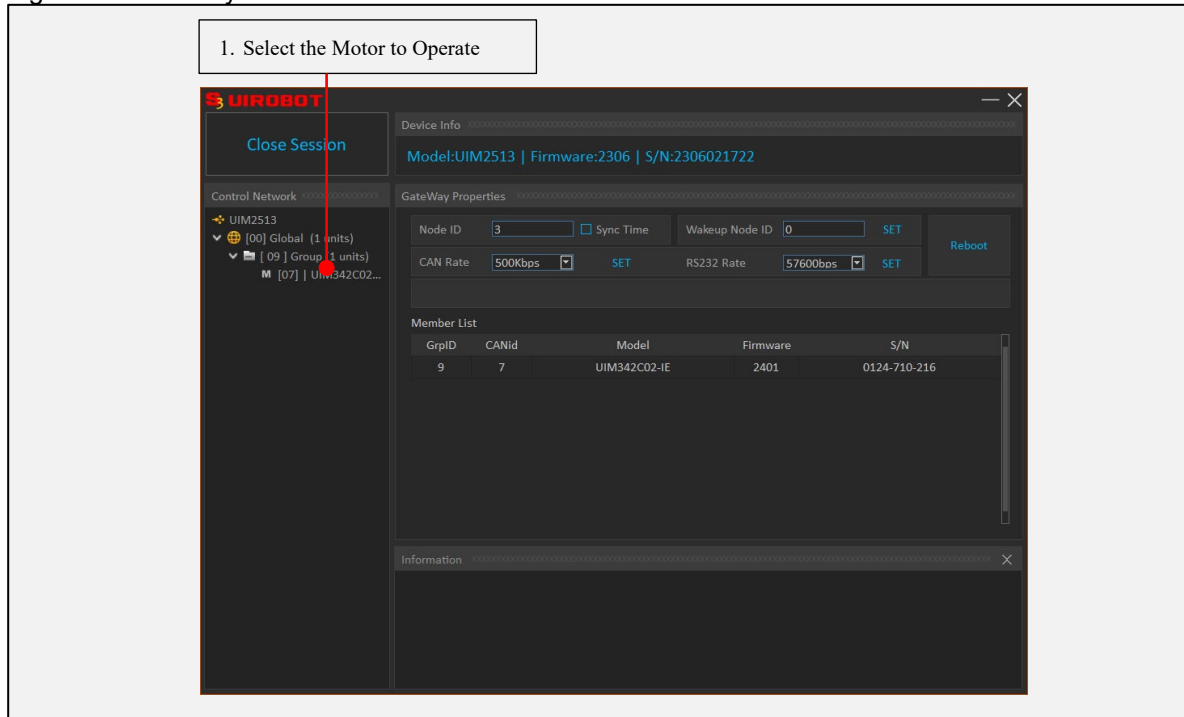
Figure 0-7: Gateway selection screen 2



UIM342 / UIM342S / UIM342XS

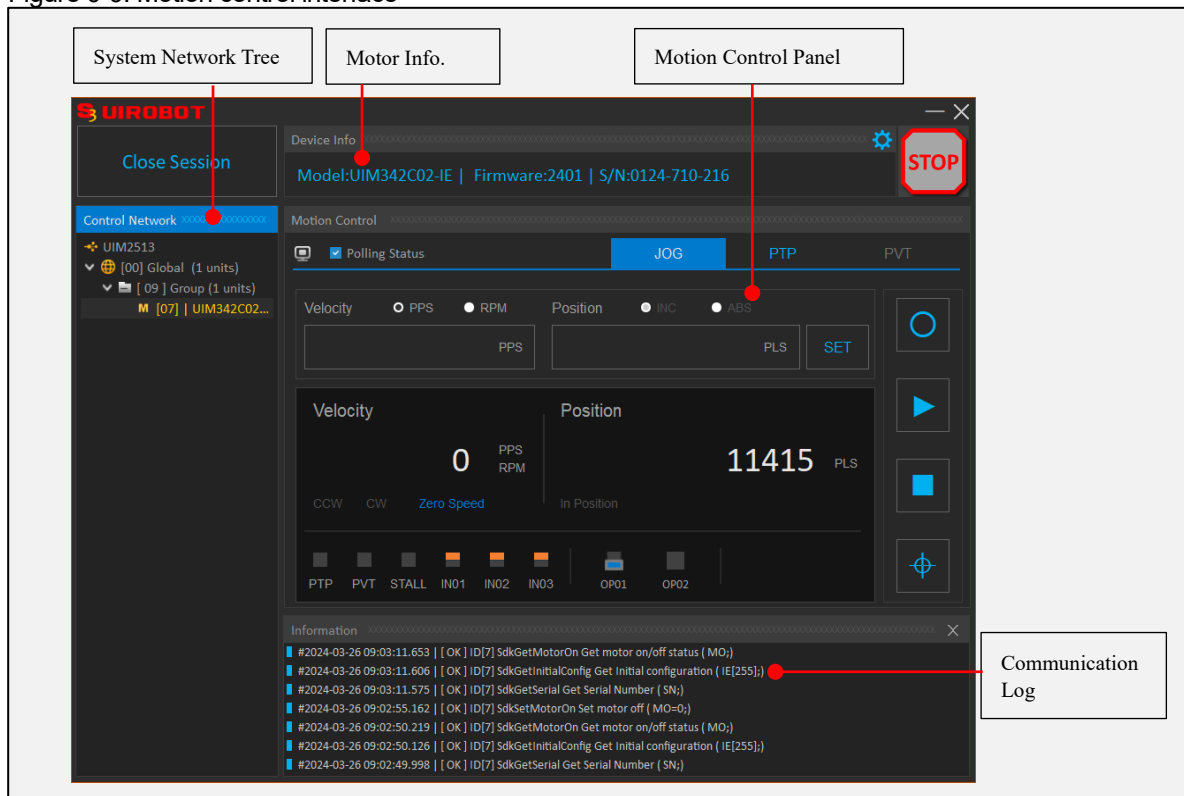
- 4) Select the UIM342C02-AB to operate in the device list on the left.

Figure 0-8: Gateway information screen



- 5) Set motion parameters and operate the motor to move.

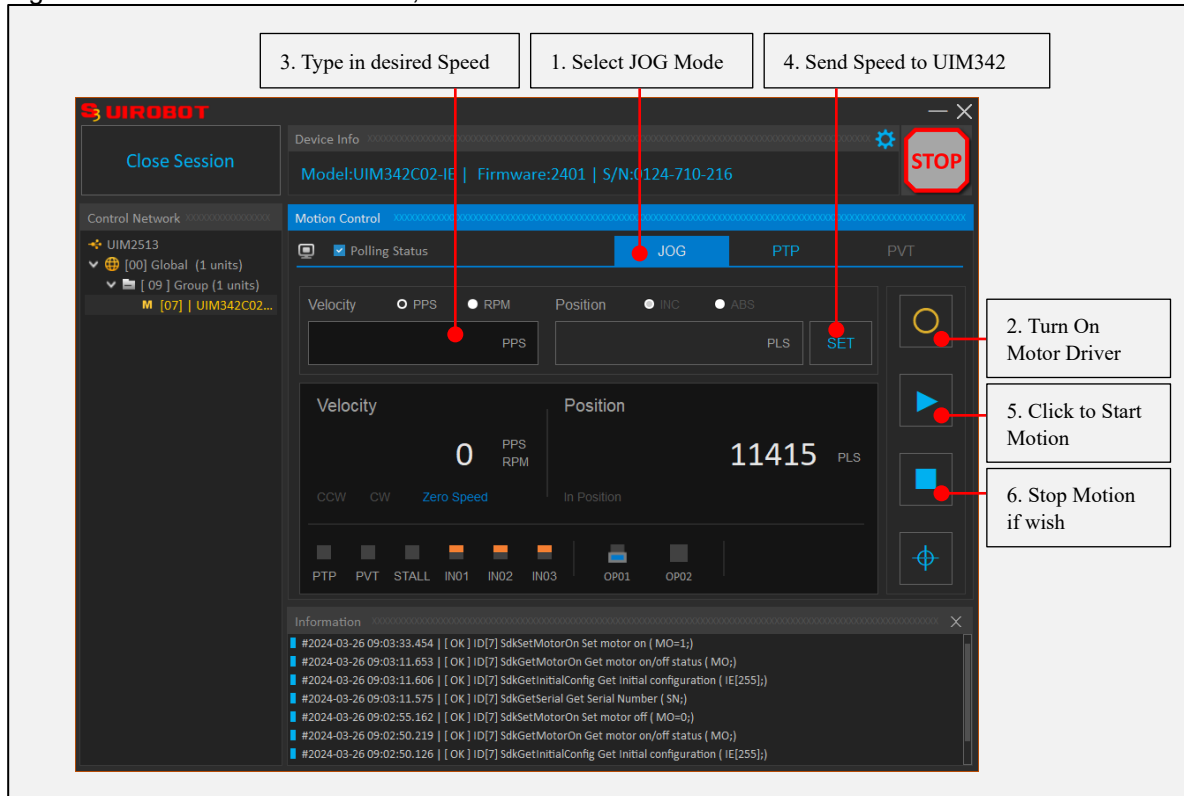
Figure 0-9: Motion control interface



For JOG Mode, follow steps 1 ...6 below to control the motor rotation.

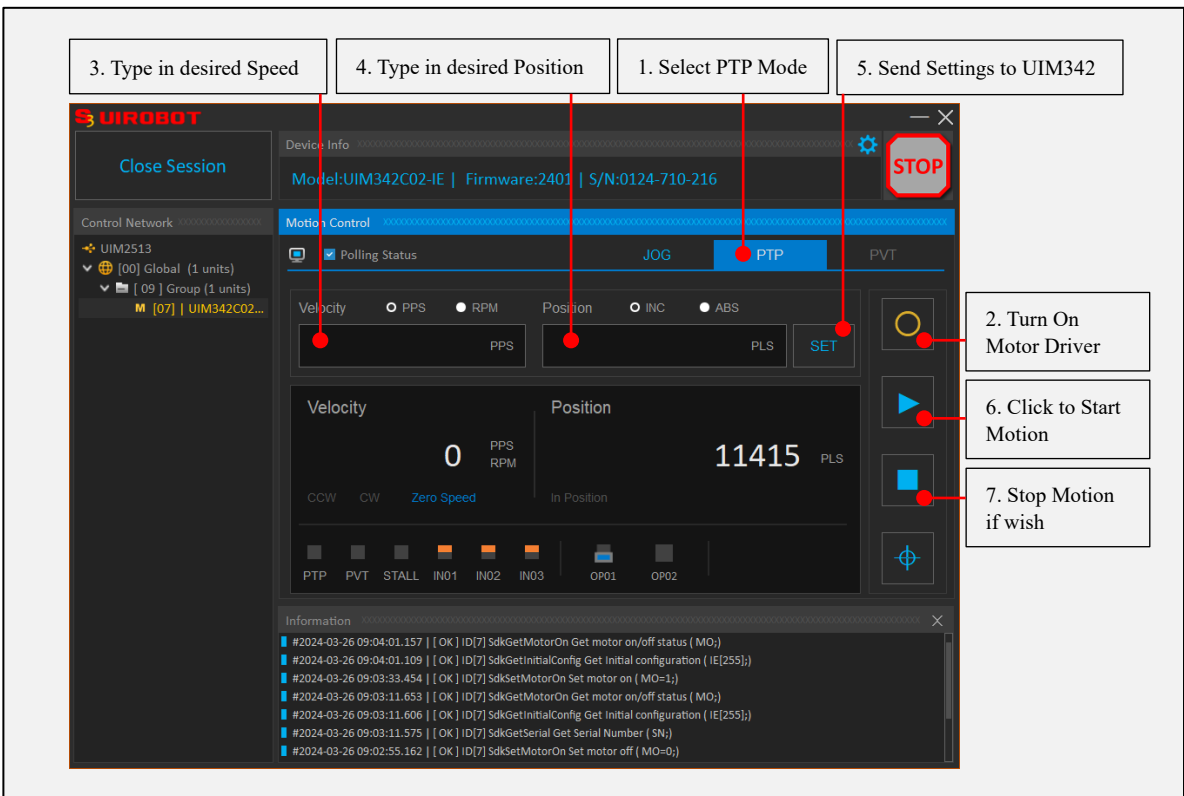
Motion Controller with CAN Interface

Figure 0-10: Motion control interface, JOG mode



For PTP Mode, follow steps 1 to 5 below to control the motor rotation.

Figure 0-11: Motion control interface, PTP mode



SPECIFICATIONS

Absolute Maximum Ratings

Ambient temperature under bias.....-40 °C to 85 °C
Storage temperature -65 °C to +150 °C
Voltage on V+ with respect to V- (UIM342XSAB / 342SAB)..... 10 V to 30 V
Voltage on V+ with respect to V- (UIM342AB)..... 12 V to 50 V
Voltage on input ports with respect to V- -0.3 V to 5.3 V
Maximum current sourced by output port..... 100 mA
CANH /CANL voltage with respect to V-..... - 40 V to + 40 V

Stresses above those listed under “Absolute Maximum Ratings” may cause permanent damage to the device. This is a stress rating only and functional operation of the device at those or any other conditions above those indicated in the operational listings of this specification is not implied. Exposure to maximum rating conditions for extended periods may affect device reliability.

Operating Conditions (at ambient temperature 25 °C)

	Specification		
	UIM342XS	342S	UIM342
Supply Voltage (DC)	12V– 28V	12V– 28V	16V– 48V
Driver Capacity	Peak 1 / 2 / 4 / 8 Amp		
Current Control	PWM constant current		
Micro-Stepping	1/2/4/8/16/32/64/128		

Operating Environment

Cooling	Free Air
Environment	Avoid dust, oil mist and corrosive gases
Operating temperature	-40 °C ~85 °C
Humidity	< 80% RH, no condensation, no frost
Vibration	3 G Max
Storage	-65 °C ~ 150 °C

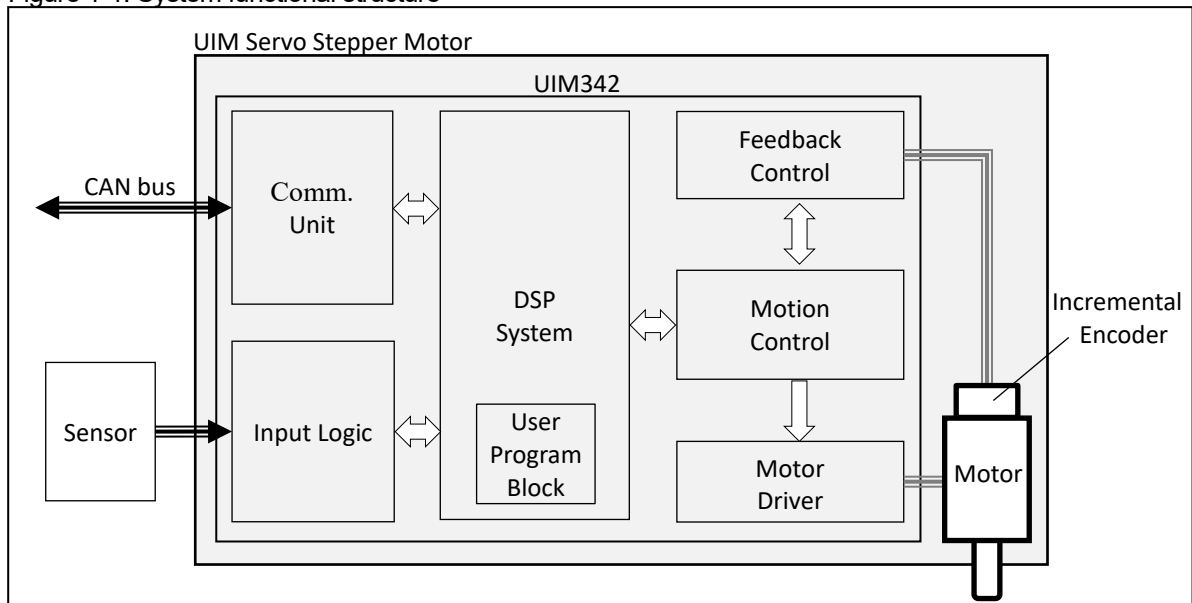
Communication Interface

CAN	Active CAN 2.0
CAN Physical	Two-wire system, CANH, CANL, twisted pair
CAN Driver	Max. 1 Mbps Meets ISO-11898 standard physical layer requirements.

1.0 Introduction

UIM342 motion controller can realize open-loop control or closed-loop control. Its main functional modules include: Communication Unit, Motion Control Unit, Motor Driver, Feedback Control Unit, Input Logic Control and DSP System (processing instruction / reply / notification, and coordinating other functional modules, executing preloaded user programs and real-time status notifications, etc.). All control loops are completed within 1 millisecond.

Figure 1-1: System functional structure



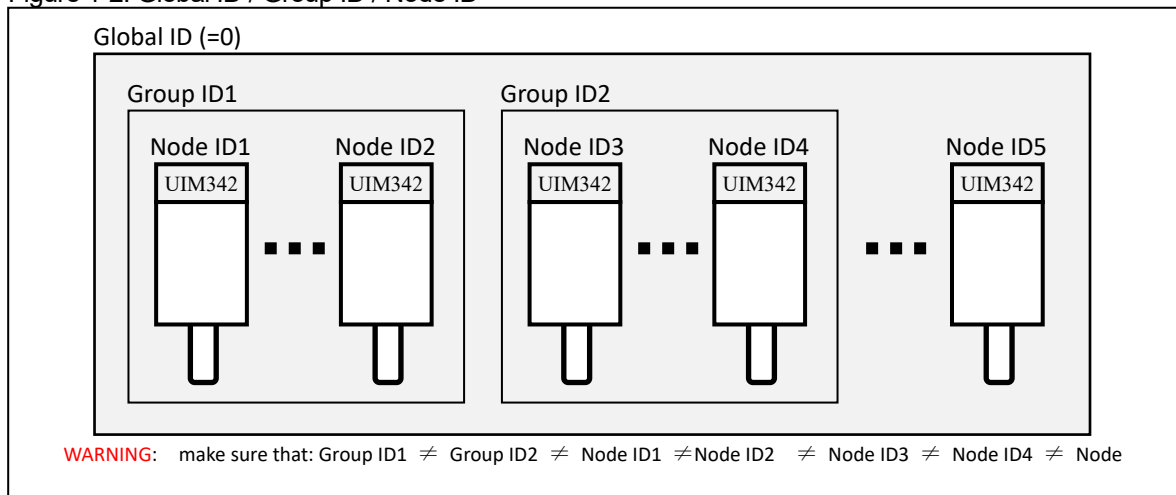
UIM342 / UIM342S / UIM342XS

1.1 Communication

Active CAN 2.0 B hardware and software is used, and CAN bit rate is configurable from 125 K to 1000 Kbps by instructions.

To improve the coordination performance of multi-modules, UIM342 supports 3 types of CAN ID: Global ID (=0), Group ID and Node ID. The Group ID and Node ID are configurable via instructions. UIM342 will accept the message that has a CAN ID matches any one of the three IDs.

Figure 1-2: Global ID / Group ID / Node ID



1.2 Motion Control

Contains hardware and software for speed control and position control in open-loop or closed-loop modes.

In addition of normal functions like speed, relative position, absolute position, UIM342 also provides functions such as backlash compensation and automatic reciprocating oscillation to facilitate medical device's needs.

1.3 Motor Driver

Supports 1/2/4/8/16/32/64/128 micro-stepping, with almost no vibration/noise when running at low speed.

In the low-speed range, compared with AC and DC servo systems, the UIM342 Servo Stepper Motor has higher torque, smaller size, and excellent cost effectiveness.

1.4 Input Logic Control

UIM342 supports 3 digital inputs. User can configure the actions to be performed on the logic level change.

There are total 6 motion parameter sets. Each input port has 2 sets of motion parameters, one for rising edge, and another for falling edge. Each motion parameter set includes: acceleration AC and deceleration DC, +/- speed SP, +/- displacement PR, +/- position PA.

There are 13 functional behaviors that can be attached to the rising/falling edge of the inputs:

- 1) Turn Motor Driver Off, Free wheel.
- 2) Emergency stop (using SD).
- 3) Decelerating to stop (using DC).

- 4) Set origin then go reversed relative position (using |PR|, SP, AC, DC).
- 5) Set origin then Emergency stop (using SD).
- 6) Set origin then decelerate to stop (using DC).
- 7) Reversed Jog (using |SP|, AC, DC).
- 8) Jog (using SP, AC, DC).
- 9) Go reverse relative position (using |PR|, SP, AC, DC).
- 10) Go relative position (using PR, SP, AC, DC).
- 11) Go absolute position (using PA, SP, AC, DC) .
- 12) Turn Motor Driver Off (Free wheel) + Lock Down the control system (i.e inquiry only).
- 13) Emergency stop (using SD) + Lock Down the control system (i.e inquiry only).

1.5 Logic after Motor Stall

After “motor stall” is detected, by default, the motor shaft will be stopped and locked.

However, set “IL[16] =1” will change the logic to “turn motor driver off (freewheel)” after a “stall” situation is detected.

1.6 Real Time Notification

UIM342 can automatically send messages to the user after detecting the occurrence of a preset event. The time from detection to event occurrence to feedback is less than 1 millisecond.

UIM342 supports real-time notification for following events:

- Reach desired position (in PTP mode),
- Rising edge and falling edge of input 1, 2, and 3,
- Detection of motor stall, and
- Other alarm/warnings.

1.7 System development SDK

The UIM342 motion controller features a simple, highly fault-tolerant instruction set. If an invalid command is entered, the controller returns an error message to the host and ignores the instruction to prevent unintended operations.

UIROBOT provides a free System Development Kit (SDK), including C++ and C# demonstration source code and sample control software for Visual Studio.

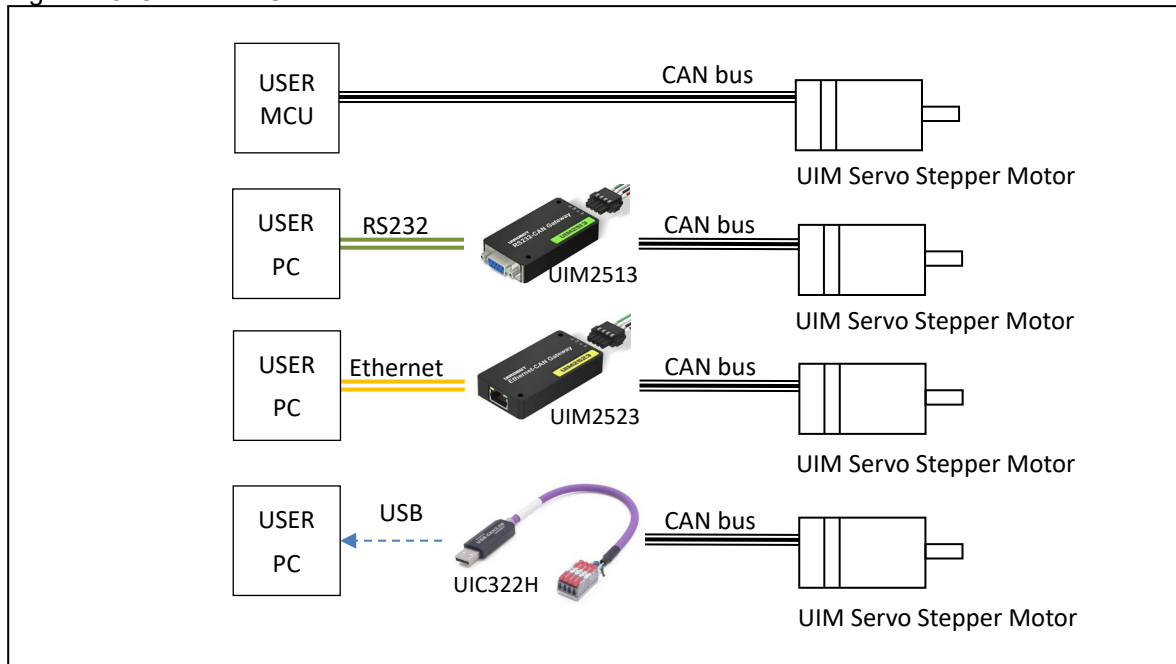
Additionally, UIROBOT offers customization services for developing user-specific control system solutions.

1.8 Connect to User Devices

There are 4 ways to connect the UIM342 to the user devices, as shown in Figure 1-3:

- 1) Directly through CAN Bus if the user controller has a CAN port;
- 2) Using the UIM2513 RS232-CAN gateway (sold separately);
- 3) Using the UIM2523 Ethernet-CAN gateway (sold separately).
- 4) Using the UIC322(H) USB-CAN Adapter (sold separately);

Figure 1-3: Connect to User Devices



2.0 Protocol

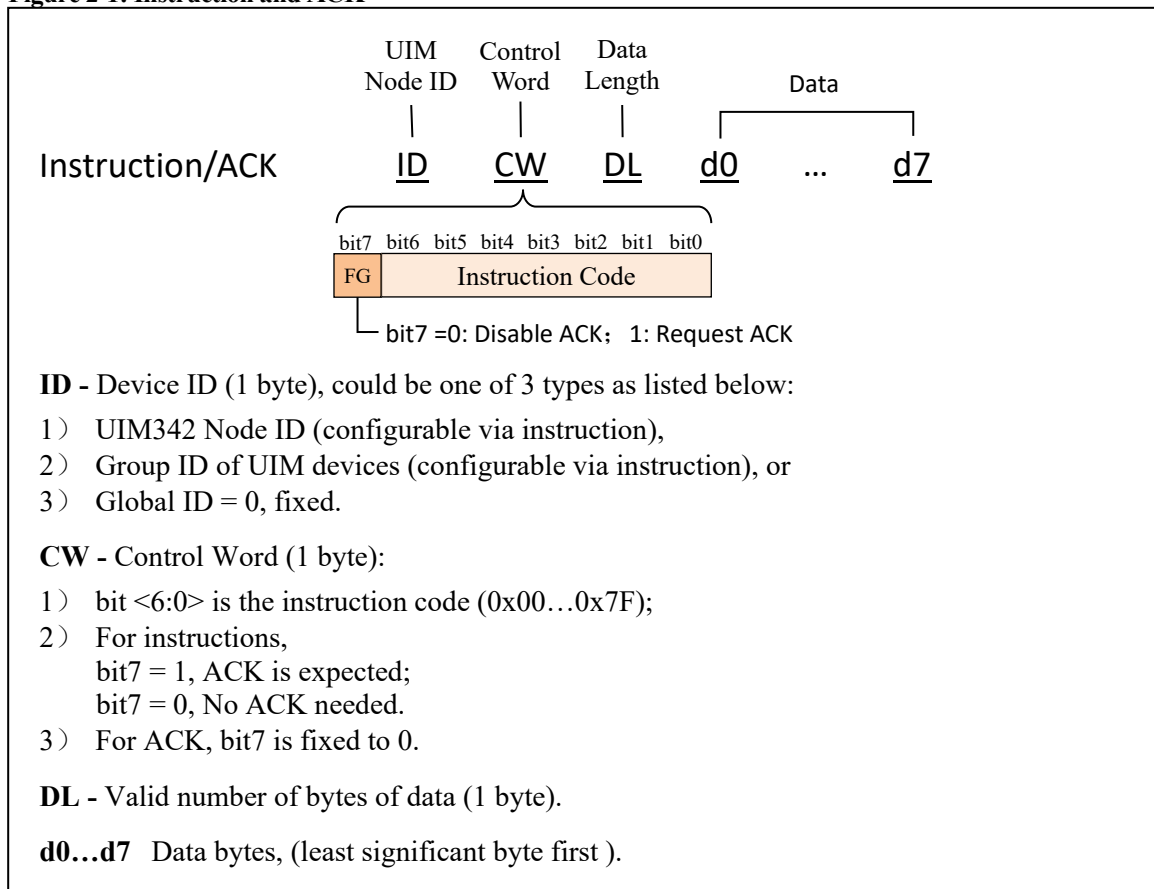
This section introduces the instructions, reply / ACK, error reporting, and real-time status and alarm notifications supported by UIM342.

2.1 Instruction and Reply (ACK)

Instruction is the commanding message sent from the user computer to the UIM342. Before receiving an instruction, UIM342 will not act.

ACK is a feedback sent from UIM342, after receiving an instruction. For setup instruction, the ACK message is typically a repeat of the instruction; for query instruction, the ACK message provides the queried data.

Figure 2-1: Instruction and ACK



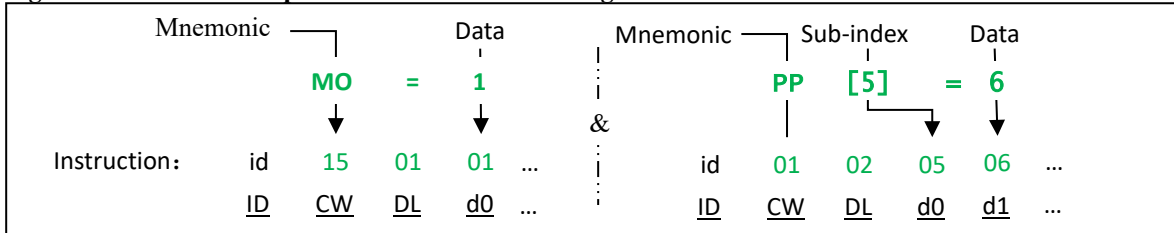
2.2 Mnemonic

For user convenience, UIM products provide a set of mnemonics used to represent various instruction codes. For example:

Mnemonic	CW	Function
MO	0x15	Turn on/off the motor driver

The relation between mnemonic expressions and instructions is shown below:

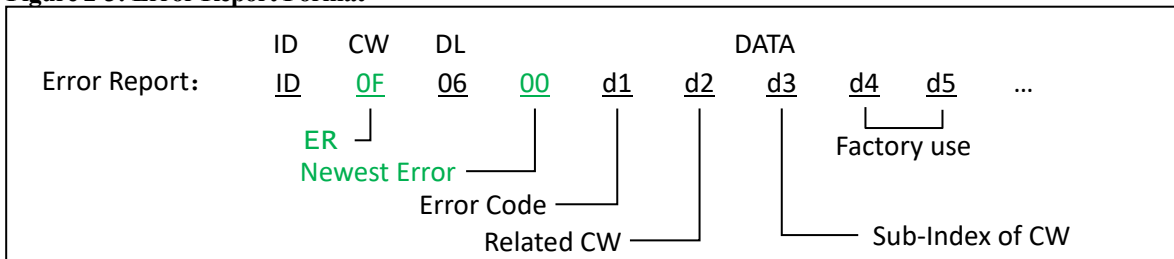
Figure 2-2: Mnemonic Expression & Instruction Message



2.3 Error Report

When UIM342 receives a wrong instruction, it will not execute the instruction, instead, it will reply an error message. Meanwhile, users can use **ER[i]** to get the latest error and the previous 7 historical errors. The format of an error message is listed below:

Figure 2-3: Error Report Format



d1 Error Code.

Error Code for UIM342	Description
0x32	Instruction Syntax error
0x33	Instruction Data error
0x34	Instruction Sub-Index error
0x3C	SD value is less than DC value
0x3D	Current instruction is not allowed when motor is running
0x3E	BG is not allowed when motor driver is OFF
0x3F	BG is not allowed during emergency stopping
0x41	OG is not allowed when motor is running

d2 CW related to the error, e.g., d2 = DI indicates an error on previous DI instruction.

d3 Sub-Index of the CW related to the error, e.g., d2= PP; d3 = 5 indicates an error on previous PP[5] instruction.

d4, d5 Factory use, don't care.

Motion Controller with CAN Interface

2.4 Real-Time Status and Alarm Notification

Real-time notifications are messages the UIM controller automatically sends to the user device, including execution status, alarms, and I/O port level change notifications.

UIM342 is equipped with a notification/information enable configuration register. Some real-time notifications can be enabled / disabled by setting this register. The notification enable register is read and written by the instruction `IE[i]`. Refer to “5.3 `IE[i]` Information Enable” for details.

The real-time status notification format sent by UIM342 to the user is as follows:

Figure 2-4: Real-time Notification

Real Time Notification	<u>ID</u>	<u>5A</u>	<u>DL</u>	<u>d0</u>	<u>d1</u>	...	<u>d7</u>
------------------------	-----------	-----------	-----------	-----------	-----------	-----	-----------

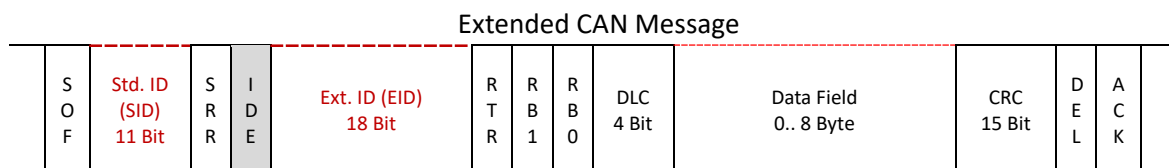
Types of notifications are shown in the table below: (The message examples are all assumed to be produced by site 5 (ID=5))

d0	d1	Type	Notification message example												Remark
-	-	-	ID	CW	DL	d0	d1	d2	d3	d4	d5	d6	d7	-	
0x00	0x0A	Alarm	05	5A	05	00	0A	00	00	00	00	00	00	Emergency stop and lock	
0x00	0x16	Alarm	05	5A	05	00	16	00	00	00	00	00	00	Acc. / Dec. over limit	
0x00	0x17	Alarm	05	5A	05	00	17	00	00	00	00	00	00	Speed over limit	
0x00	0x18	Alarm	05	5A	05	00	19	00	00	00	00	00	00	Exceed Lower Bumping Limit	
0x00	0x19	Alarm	05	5A	05	00	18	00	00	00	00	00	00	Exceed Upper Bumping Limit	
0x00	0x1A	Alarm	05	5A	05	00	1A	00	00	00	00	00	00	Exceed Lower Working Limit	
0x00	0x1B	Alarm	05	5A	05	00	1B	00	00	00	00	00	00	Exceed Upper Working Limit	
0x00	0x1D	Alarm	05	5A	05	00	1D	00	00	00	00	00	00	Motor Stall Detected	
0x00	0x1E	Alarm	05	5A	05	00	1E	00	00	00	00	00	00	Encoder Error	
0x00	0x1F	Alarm	05	5A	05	00	1F	00	00	00	00	00	00	Encoder Battery Low	
0x01	0	Status	05	5A	01	01	00	00	00	00	00	00	00	Input 1 falling edge detection	
0x02	0	Status	05	5A	01	02	00	00	00	00	00	00	00	Input 1 rising edge detection	
0x03	0	Status	05	5A	01	03	00	00	00	00	00	00	00	Input 2 falling edge detection	
0x04	0	Status	05	5A	01	04	00	00	00	00	00	00	00	Input 2 rising edge detection	
0x05	0	Status	05	5A	01	05	00	00	00	00	00	00	00	Input 3 falling edge detection	
0x06	0	Status	05	5A	01	06	00	00	00	00	00	00	00	Input 3 rising edge detection	
0x29	0	Status	05	5A	08	29	00	00	00	p0	p1	p2	p3	PTP positioning completed, [p3:p0] = current position	

2.5 Direct CAN Communication

When directly communicating with UIM342 devices via CAN Bus, the user CAN controller needs to be configured as follows:

- 1) TQ number, use one of the following options:
 - 1 M bps use 8 TQ;
 - 800 K bps use 10 TQ;
 - Any other bps, use 16 TQ.
- 2) Synchronous jump: 1 TQ.
- 3) Sample once at each sampling time.
- 4) Phase Segment 2: 2 TQ.
- 5) Filter and MASK are configured to receive all messages.
- 6) Set SRR = 1; IDE = 1; RTR = 0; RB1 = 0; RB0 = 0;



EID, SID, DLC, and data0...data7 of the CAN message should be processed as follows:

Sending instructions to UIM342:

```
SID = ((ID<<1) & 0x003F) | 0x0100; //ID (Consumer, UIM's Address)
EID = (((ID<<1) & 0x00C0)<<8) | CW; //CW (Control Word)
DLC = DL; //DL (Effective Data Length)
CANTX_B0 = d0;
CANTX_B1 = d1;
CANTX_B2 = d2;
CANTX_B3 = d3;
CANTX_B4 = d4;
CANTX_B5 = d5;
CANTX_B6 = d6;
CANTX_B7 = d7;
```

Note: Some controllers may use 32bits CAN-ID, which = (SID<<18) | EID;

Parsing received messages from UIM342:

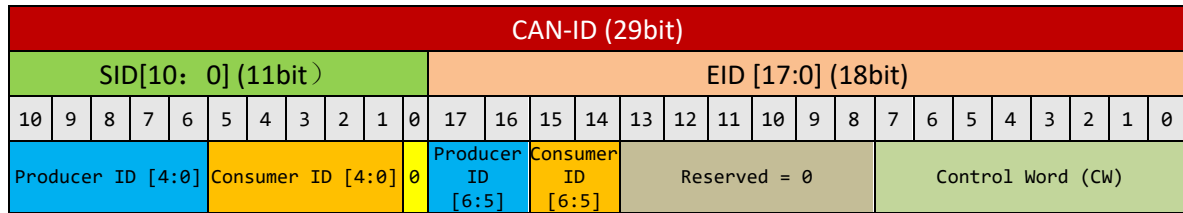
```
ID = ((EID>>11)&0x0060)|((SID>>6)&0x001F); //ID (Producer, UIM's Address)
CW = EID&0x00FF; //CW (Control Word)
DL = DLC&0x000F; //DL (Effective Data Length)
d0 = CANRX_B0;
d1 = CANRX_B1;
d2 = CANRX_B2;
d3 = CANRX_B3;
d4 = CANRX_B4;
d5 = CANRX_B5;
d6 = CANRX_B6;
d7 = CANRX_B7;
```

Note: Some controllers may use 32bits CAN-ID, which = (SID<<18) | EID;

Motion Controller with CAN Interface

To help further understanding the CAN protocol (i.e., SimpleCAN3.0) used to control the UIM342, more details and examples are listed below.

CAN-ID for SimpleCAN3.0



Producer ID	ID of the controller (Master); For user master controller, should use ID = 4.
Consumer ID	ID of UIM342 (Initial ID = 5, range 5...127), can be modified via Instructions.
Control Word	Control Word for UIM342; For instance, CW = 0x95 is to enable the motor driver.

Calculation of SID:

$SID = ((Consumer\ ID \ll 1) \& 0x003F) \mid 0x0100$

Calculation of EID:

$EID = (((Consumer\ ID \ll 1) \& 0x00C0) \ll 8) \mid CW$

In case of your controller only supports the CAN-ID (32-bit) format, calculate as follows:

$CAN-ID = SID \ll 18 \mid EID$

Example

Using a CAN master device with ID 4 to enable (i.e., MO=1;) a UIM342 motor driver with ID 5

Producer ID = 4

Consumer ID = 5

CW = 0x95 (MO)

Data = 1;

Calculate SID: $SID = ((5 \ll 1) \& 0x003F) \mid 0x0100 = 0x010A;$

Calculate EID: $EID = (((5 \ll 1) \& 0x00C0) \ll 8) \mid CW = 0x0095;$

Calculate CAN-ID: $CAN-ID = 0x010A \ll 18 \mid 0x0095 = 0x04280095;$

DLC = 0x01;

Data0 = 0x01;

Send following message from user controller to UIM342:

CAN-ID (hex)	Date Field(hex)
04280095	01

If the user computer needs to specify DLC, please send following:

CAN-ID (hex)	DLC (hex)	Date Field (hex)
04280095	01	01

UIM342 / UIM342S / UIM342XS

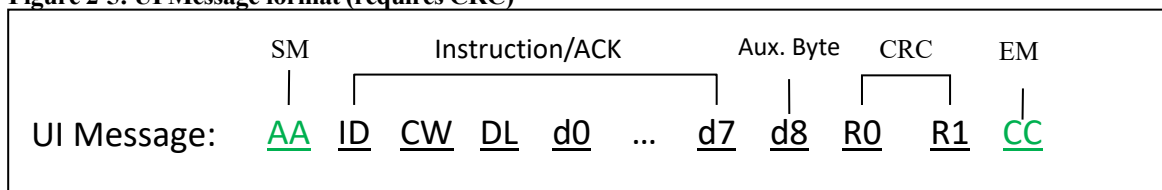
More examples of CAN message used to control the UIM342 motors are listed below.

Can-ID (hex)	Data (hex)	Function
04280095	00	MO = 0, set driver off
04280095	01	MO = 1, set driver on
0428009D	10 27	JV = 10000pps, set jog speed 10000pps
0428009D	F0 d8 FF FF	JV = -10000pps, set jog speed -10000pps
04280096	No data	BG, set begin motion
04280097	No data	ST, set stop motion
0428009F	80 0C	PR = 3200
0428009F	80 F3 FF FF	PR = -3200
042800A0	80 0C	PA = 3200
042800A0	80 F3 FF FF	PA = -3200
0428009E	80 0C	SP = 3200

2.6 RS232 / Ethernet Gateway

Gateway such as UIM2513, UIM2523 Messages send / receive are all UI Message format.

Figure 2-5: UI Message format (requires CRC)



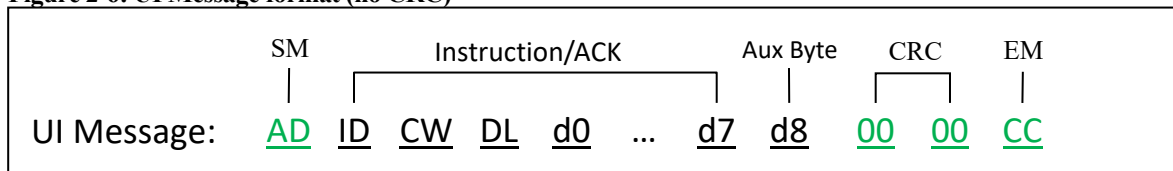
SM Start of Message. If one needs CRC, using 0xAA, else using 0xAD instead,

d8 Auxiliary byte, don't care,

R1:R0 RTU CRC16, range covers AA...d8, refer to Appendix- 1 for calculation source code,

EM End of Message; is fixed to 0xCC.

Figure 2-6: UI Message format (no CRC)



Notice: UI Message is in Hex Format, and the length of the message is fixed to 16;

3.0 Motion Control

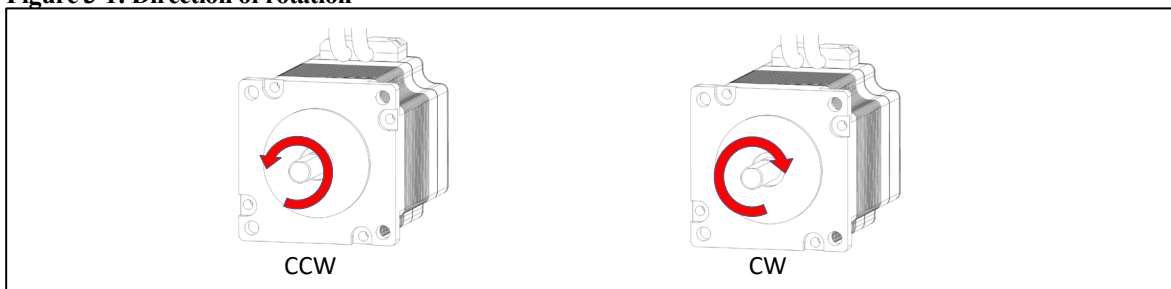
UIM342 motion controller supports speed control (JOG) mode and point to point (PTP) position control mode. User can Set, Get and Store motion parameters through instructions. Meanwhile, user can also receive the working status of the motor through real-time notification.

This chapter briefs the motion control related functions of UIM342.

3.1 Direction of Rotation

Facing the output shaft, direction of the motor rotation is defined as CW if the shaft rotates clockwise, and CCW if the shaft rotates count clockwise.

Figure 3-1: Direction of rotation



Set “IC[1]=1” to configure the position counter increase when the motor is running in CCW direction (Factory default). Otherwise, set “IC[1]=0” to configure the position counter increase when the motor is running in CW direction.

3.2 Motion Control Modes

- **Speed control (JOG)**

UIM342 controls the motor speed to reach the desired speed set by the user through an acceleration or deceleration process.

The JV instruction sets the desired speed value, and the sign (+/-) of the value determines the direction of the rotation. In addition, the AC instruction sets the acceleration rate and the DC instruction sets deceleration rate.

After a JV instruction, the BG instruction must be set to start the movement. In this manner, multiple UIM342 controllers can be set JV one by one, and start moving at exactly the same time via sending BG to the group ID or the global ID.

- **Position control (PTP)**

The **UIM342** controls motor speed and displacement, stopping when the desired position is reached. It adjusts the speed to its maximum while ensuring precise position accuracy.

- The **SP** instruction sets the target speed for position control.
- The **PA** instruction defines the absolute position.
- The **PR** instruction defines the relative position from the current location.

After setting the PA/PR instruction, use the **BG** instruction to initiate movement. Multiple UIM342 controllers can be programmed with PA/PR and started simultaneously by sending the BG command to a specific group ID or the global ID.

Note: The motor's actual speed and direction are determined by the position deviation (desired position - actual position). If the position deviation is minimal, the motor may reach the target position before achieving the desired speed.

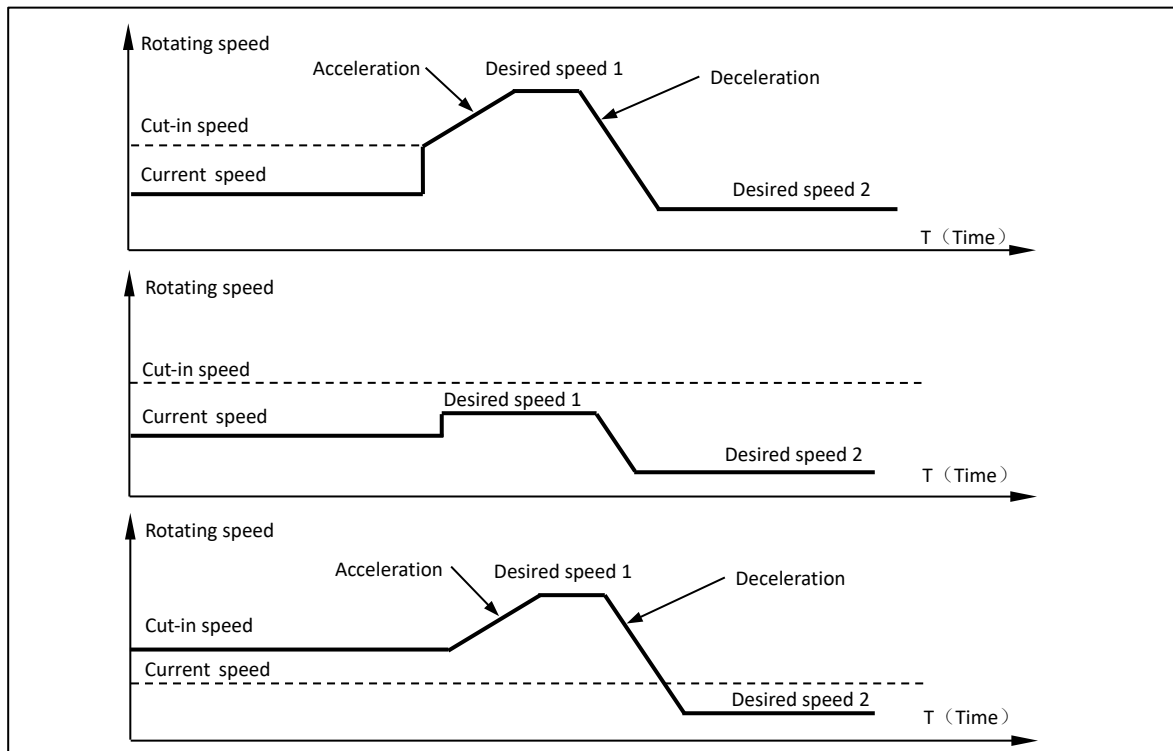
- **Interpolated Motion (PVT/PT)**

The UIM342AB controller has an Absolute Multi-turn Encoder and supports the curve interpolation motion (PVT/PT Motion). Through the PVT/PT (Position-Velocity-Time) control, the user only needs to set key control points (position, speed, and time interval), the controller will automatically generate a continuous and smooth motion trajectory between these points, ensuring that the position, speed, and acceleration are continuous throughout the motion process, avoiding sudden changes and shocks.

3.3 Acceleration, Deceleration and Cut-in speed

In order to improve the response of the motor and avoid the resonance, the acceleration (deceleration) process includes a cut-in speed.

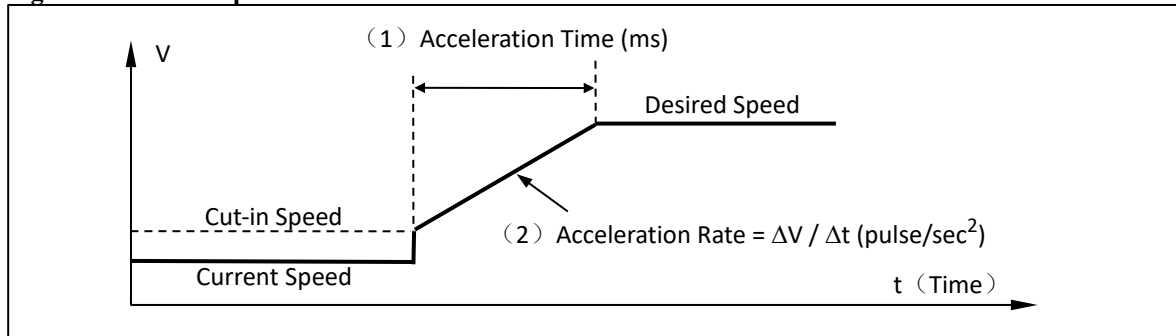
Figure 3-2: Uniform acceleration (deceleration) speed and cut-in speed



Motion Controller with CAN Interface

Instructions AC, DC, SS are used to set acceleration, deceleration and cut-in speed. Acceleration and deceleration can be defined by the time of the process, or the rate of acceleration and deceleration. The two definitions are shown in the figure below.

Figure 3-3: AC / DC parameter



3.4 Backlash Compensation

Mechanical systems like screw-nut or rack-and-pinion transmissions often experience backlash, which causes a delay when reversing direction. This leads to accumulated error that must be compensated.

UIM342 addresses this issue by providing backlash compensation. The compensation value can be adjusted using the BL instruction, with a range from 0 to 65535 pulses. This helps to minimize errors and improve system accuracy.

3.5 Closed Loop / Open Loop Control

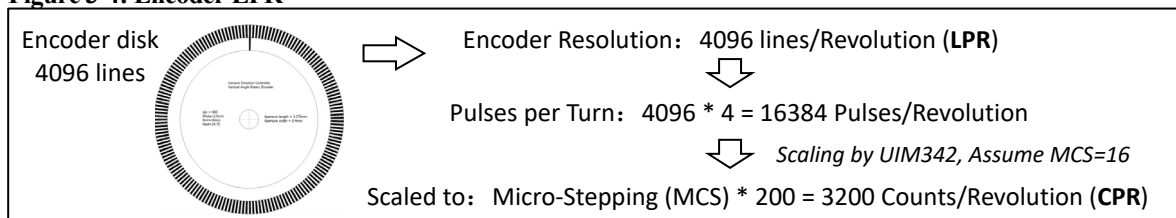
The UIM342 motor controller supports both open-loop and closed-loop control:

- **Open-Loop Control:** Set IC[6]=0 to enable this mode, which operates without feedback.
- **Closed-Loop Control:** Set IC[6]=1 to activate this mode, where a quadrature encoder provides feedback for precise displacement control.

In closed-loop mode, the quadrature encoder ensures accurate motor positioning by delivering displacement feedback.

LPR (Lines per Revolution) refers to the number of lines engraved on the encoder disk. For each full motor revolution, the encoder generates 4 times the LPR value in pulses.

Figure 3-4: Encoder LPR



CPR (counts per revolution) is derived from the encoder's 4xLPR according to the selected micro-step setting (MCS). Numerically, CPR equals the number of step-pulses the motor receives for one full revolution at that MCS, so using CPR eliminates any conversion between encoder counts and motor pulses.

UIM342 uses CPR to scale encoder counts for position comparison.

For a 1.8 deg stepper (200 full steps/rev), $CPR = 200 \times MCS$.

Examples:

- MCS = 16 → CPR = 200 × 16 = 3,200 counts/turn
- MCS = 64 → CPR = 200 × 64 = 12,800 counts/turn

3.6 Pulse Rate vs Speed

Following is the equation to calculate the required pulse rate (PPS, pulse per second):

$$RPM = 60 \times \frac{PPS}{MCS \times 200}$$

or

$$PPS = \frac{RPM \times MCS \times 200}{60}$$

Where:

MCS - Micro-stepping Resolution, e.g., 1, 2, 4, 8, 16, 32, 64, etc.

RPM - Revolutions per Minute.

PPS - Pulses per Second, the speed value sent to the motor.

3.7 Stall Detection

Stall detection identifies when the motor fails to follow commanded motion, typically due to overload, binding, insufficient torque, excessive acceleration, or sudden load changes. The controller continuously compares drive pulses sent to the motor with position feedback from the encoder.

- **Stall Tolerance (QE[1])**

Stall Tolerance defines the maximum allowed difference, in step pulses, between:

- Total drive pulses issued to the motor driver.
- Total feedback pulses from the encoder.

This threshold is configurable via QE[1].

- **Stall Event Reaction (IC[16])**

If position error $\geq$ Stall Tolerance, the UIM342 asserts a stall event and executes one or more user-defined actions:

- Stop motion immediately, then either lock the shaft or freewheel, per IC[16] settings.
- Signal an alarm output or set a status flag for external systems.

Reaction method should be selected according to safety, application, and operational requirements.

- **Reset position error / stall event**

The stall event and its associated position error are automatically cleared once the stall is handled or the motor driver is turned off (MO=0;).

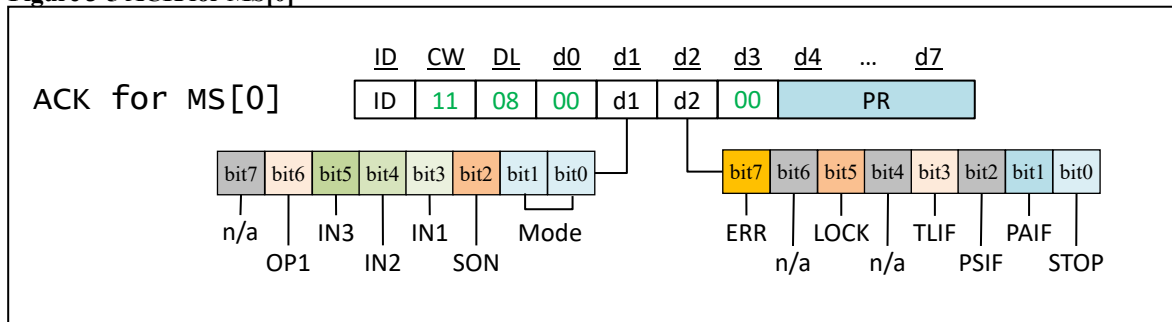
3.8 Acquiring Motion Status

To facilitate quick query of motion status, the MS[i] instruction can be used as described below.

- Get Status Flags and Relative Position**

MS[0] is used to get the current motion status flags and the relative position. The ACK Message for this instruction is explained in the figure below:

Figure 3-5 ACK for MS[0]



Bit fields are described below:

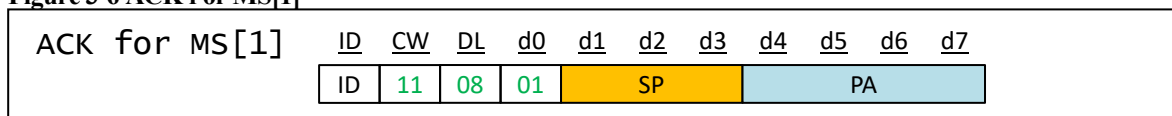
Bit field	Name	Description	Value
d1.bit<1:0>	Mode	motion mode	0 = JOG; 1 = PTP
d1.bit2	SON	motor driver	0 = OFF; 1 = ON
d1.bit3	IN1	IN1 Logic Level	0 = Low; 1 = High
d1.bit4	IN2	IN2 Logic Level	0 = Low; 1 = High
d1.bit5	IN3	IN3 Logic Level	0 = Low; 1 = High
d1.bit6	OP1	OP1 Logic Level	0 = Low; 1 = High
d1.bit7	n/a		0
d2.bit0	STOP	Motor is in stationary	0 = NO; 1 = YES
d2.bit1	PAIF	Motor is in position	0 = NO; 1 = YES
d2.bit2	PSIF	PVT Stopped	0 = NO; 1 = YES
d2.bit3	TLIF	Motor stall is detected	0 = NO; 1 = YES
d2.bit4	n/a		0
d2.bit5	LOCK	System is locked down	0 = NO; 1 = YES
d2.bit6	n/a		0
d2.bit7	ERR	System error is detected	0 = NO; 1 = YES
d3	n/a		00
d7:d6:d5:d4	PR	Current relative position	Signed 32 bit integer, LSB received first

Note: LSB - least significant byte

- Get Speed and Absolute Position**

MS[1] is used to get the current speed and absolute position.

Figure 3-6 ACK for MS[1]



Bit fields are described below:

UIM342 / UIM342S / UIM342XS

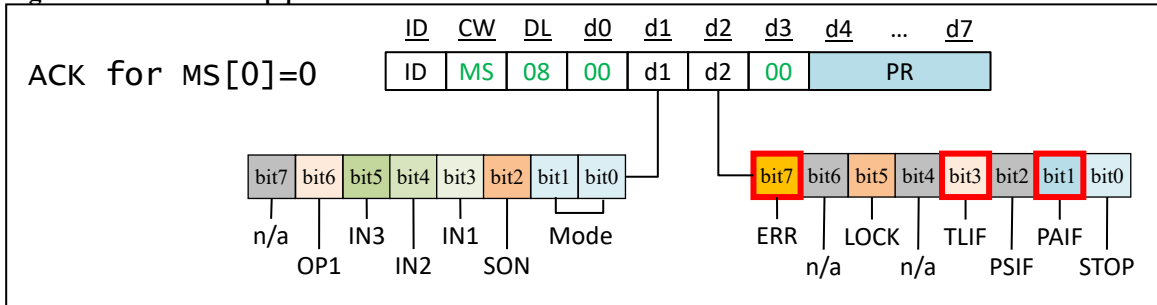
Bit field	Name	Description	Value
d3:d2:d1	SP	Current speed	Signed 24 bit integer, LSB received first
d7:d6:d5:d4	PA	Current absolute position	Signed 32 bit integer, LSB received first

Note: LSB - least significant byte

• Clear Status Flags

“MS[0]=0” is used to clear the flag bit PAIF, TLIF and ERR, and the ACK message for this instruction is explained in the figure below (values in the red boxes are cleared):

Figure 3-7 ACK for MS[0]=0



3.9 Internally Controlled Brake

The UIM342 has internal brake control circuits, powered and managed by internal control logic. It does not require an external power source or control signal.

• Control commands

- MT[5] = 0; Release the brake, free the shaft.
- MT[5] = 1; Engage the brake, lock the shaft.

• Safety Logic

The UIM342 also incorporates safety logic that helps enforce certain safety and operational rules:

Brake Status	Safety Rules	Override / Notes
Brake released	“MO=0;” is forbidden	Sending twice: ‘MO=0;MO=0;’ to force motor driver off. Ignore the first command returns error “Brake is released, offline not allowed”.
Brake engaged	“BG; ” is forbidden	Cannot override.

Users can disable the brake safety logic with “IC[8] = 0;” and re-enable it with “IC[8] = 1;”.

• Automatic release brake on power-up

To automatically release the brake at power-up, set “IC[8] = 1;”, vise-versa.

Note: Not all motors are equipped with an internally controlled brake. Before operating the brake, make sure IC[15] = 1; (IC[15] enables the using of the built-in control circuit for the brake.)

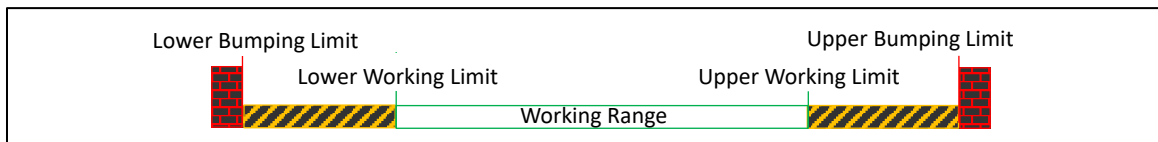
3.10 Software Limits

The UIM342 provides configurable software limits to prevent excessive motion and protect the system. These limits are stored in the LM[index] array as follows:

Motion Controller with CAN Interface

Index	Description	Action on Exceeding Limit
LM[0]	Max Working Speed	Capped at this value and Real-time Notification
LM[1]	Lower Working Limit	Emergency Stop and Real-time Notification
LM[2]	Upper Working Limit	Emergency Stop and Real-time Notification
LM[3]	Lower Bumping Limit	Emergency Stop and Real-time Notification
LM[4]	Upper Bumping Limit	Emergency Stop and Real-time Notification
LM[6]	Max Position Error	Triggers Stall Event and Real-time Notification
LM[7]	Max Acceleration / Deceleration	Capped at this value and Real-time Notification
LM[254]	Reset to default values: Lower Bumping Limit = -2^{31} ; Upper Bumping Limit = $2^{31} - 1$; Lower Working Limit = $-2^{31} + 3$; Upper Working Limit = $2^{31} - 3$;	N/A
LM[255]	Enable / Disable Software Limits	1: Enable; 0: Disable.

The relationship between bumping limits and working limits is illustrated below.



After the software limit check is enabled, the UIM342 automatically sends a real-time notification to the host controller whenever a limit point is reached. Details of these real-time notifications are provided in Section 2.4, Real-Time Status and Alarm Notification.

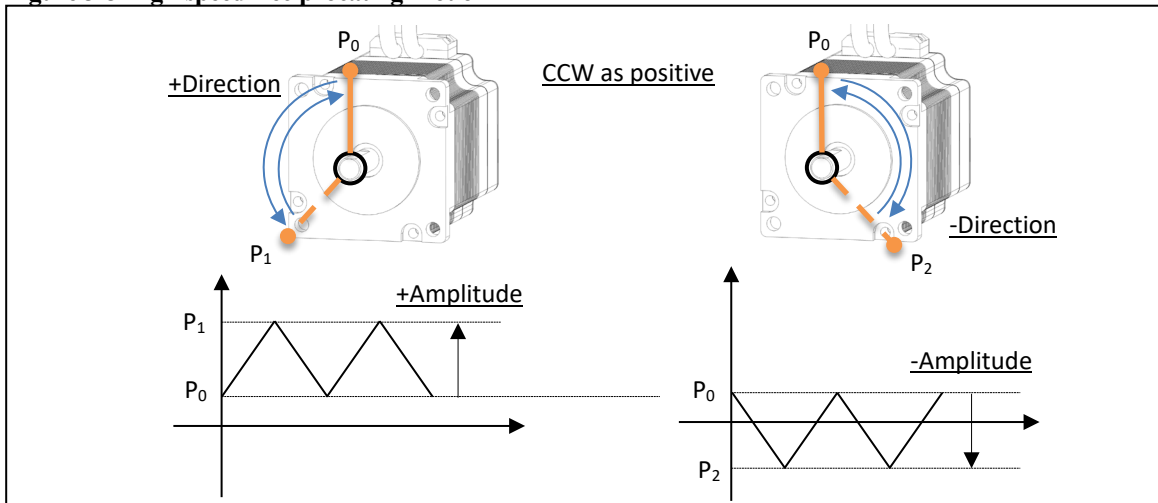
3.11 Built-in Functional Logics

Currently, the UIM342 provides two built-in functions: High-speed reciprocating motion and Fixed-angle pulse output.

• High-speed Reciprocating Motion

High-speed reciprocating motion refers to the motor performing repeated back-and-forth movements according to user-defined frequency, amplitude, and count, without any external intervention. Depending on the load, the reciprocating frequency can reach up to 50 Hz. The amplitude determines the travel range, while the sign (positive or negative) of the amplitude determines the initial movement direction. See the diagram below for reference.

Figure 3-8 High-speed Reciprocating Motion



UIM342 / UIM342S / UIM342XS

To start high-speed reciprocating motion, use the following command format:

ID	CW	DL	d0	d1	d2	d3	d4	d5	d6	d7
ID	D1	08	03	01	FRC	AMP			CNT	

- **FRC**, Frequency, range: [1...200] (= 0.25 ... 50 Hz).
Example: d2 = 0x64 (FRC=100) results in a vibration frequency of $100 \times 0.25 \text{ Hz} = 25 \text{ Hz}$.
- **AMP**, Amplitude, range: $\pm[5...8,000,000]$ pulses.
Example: d5 = 0x00, d4 = 0x03, d3 = 0xE8, results in a vibration amplitude of $0x0003e8 = 1000$ pulses.
- **CNT**, Number of cycles, range: [1...32,000].
Example: d7 = 0x00, d6 = 0x20, results in a vibration cycle of $0x0020 = 32$ times.

To stop the high-speed reciprocating motion during operation, simply send a command in the same format as above, but set **FRC** to 0.

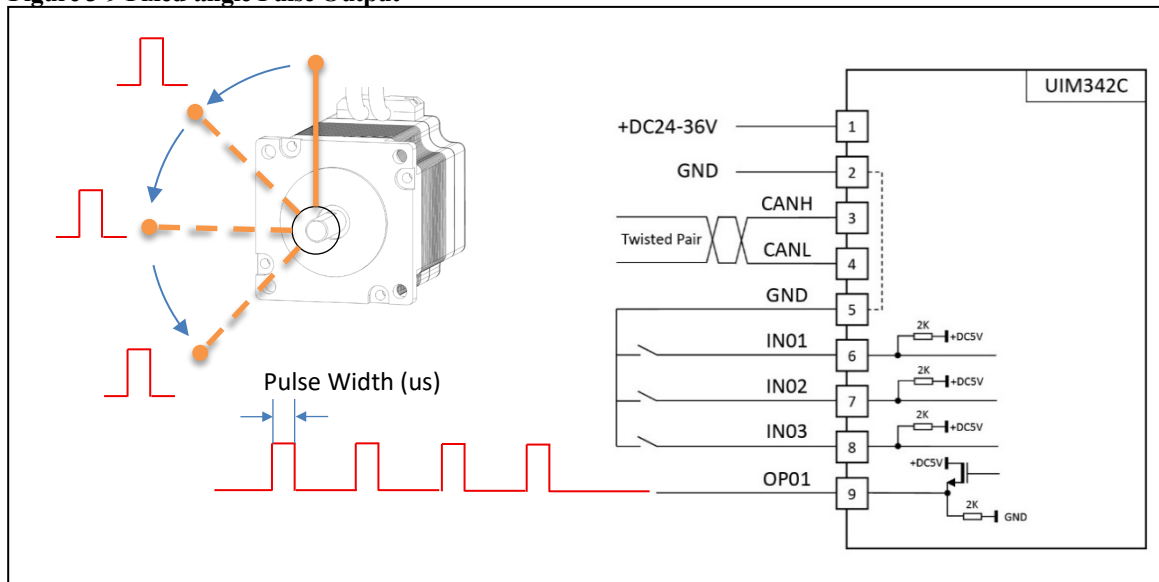
• Fixed-angle Interval Pulse Output

This function outputs a pulse each time the motor shaft rotates by a fixed, user-defined angular interval.

The angular interval is determined by encoder counts, ensuring that every movement over exactly this distance will issue one output pulse. These pulses can be used to trigger auxiliary devices such as cameras, marking systems, or measurement equipment, ensuring accurate actuation in sync with mechanical motion.

This mode differs from time-based pulse generation in that it is purely position-driven, providing high repeatability in motion-synchronized applications.

Figure 3-9 Fixed-angle Pulse Output



Users can enable this function using the following command:

ID	CW	DL	d0	d1	d2	d3	d4	d5	d6	d7
ID	D1	08	03	04	Angle			PW		00

- **Angle**, the angle interval, it is specified in **encoder counts**.

Motion Controller with CAN Interface

For UIM342, the encoder used for this function is fixed at **16,384 counts per revolution**. Therefore, the minimum angle value is: $360^\circ / 116384 = 0.02197265625^\circ$

Example: To generate a pulse every 2° at the UIM342 output port, set **Angle** = **91** counts, which means $d4 = 0x00$; $d3 = 0x00$; $d2 = 0x5B$ ($0x00005B = 91$).

- **PW**, output pulse width, range: **[20...1000] μ s**.

To disable this function, simply send the above command with **Angle** set to **0**.

Note: The values of **Angle** and **PW** will **not** be saved to EEPROM. After the motor is powered on, both **Angle** and **PW** will be initialized to **0**. Users need to send above command after motor powered up to enable this function.

4.0 I/O & Input Logic Control

The input logic control module has the following functions:

Processing and reporting the logic levels of input ports,

Executing user preset actions, when the input logic level change is detected:

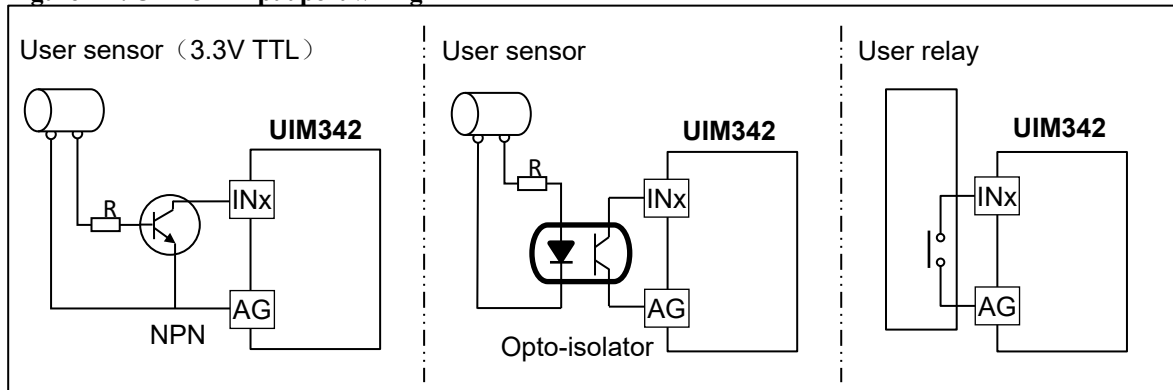
- 1) Turn off the Motor Driver (freewheel);
- 2) Emergency stop;
- 3) Decelerate until stop;
- 4) Set origin then turn around and go to relative position;
- 5) Set origin then Emergency stop;
- 6) Set origin then decelerate until stop;
- 7) Turn around and move in constant speed (Jog);
- 8) Start to move in constant speed (Jog);
- 9) Turn around and go to relative position;
- 10) Go to relative position;
- 11) Go to absolute position;
- 12) Turn off the Motor Driver (freewheel) + Lock down motion control (become read only);
- 13) Emergency stop + Lock down motion control (become read only);

4.1 Wiring the Sensor

UIM342 input ports only takes 5V TTL signal. If a 3.3V TTL sensor is to be used, an NPN transistor should be added as shown in Figure 5-1.

It is recommended to use a photoelectric isolation module between the sensor and the INx port of the controller. The photoelectric isolation module has strong anti-interference properties and is recommended for use in factory environments and situations with significant interference. Please adjusting the resistance R to ensure that the current through the isolator is around 10 mA.

Figure 4-1: UIM 342 input port wiring



4.2 Trigger type and Input Filter

The voltage on each input port can be processed with three types of trigger:

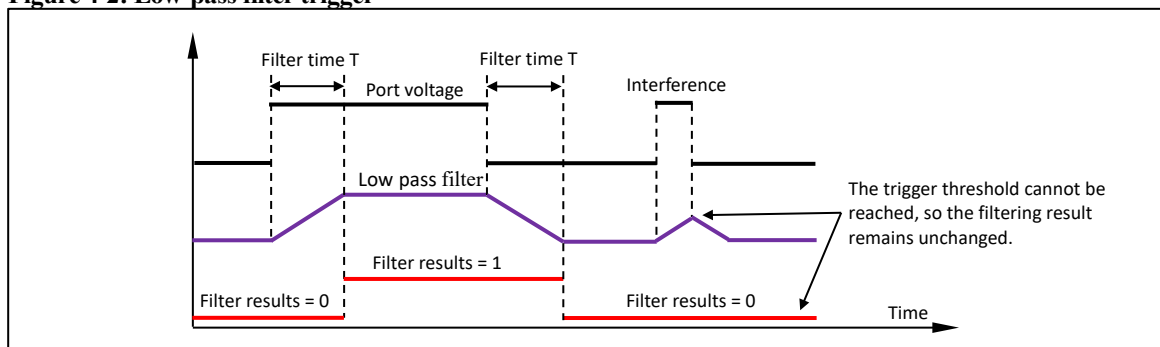
- **Continuous**

This type allows the UIM342AB continuously sampling the voltage on input ports. If a logic level change is detected, UIM342AB will immediately notified the user and execute the preset logic actions. To use this trigger, set “TG[i]=0”, i=0, 1 or 2.

- **Low-Pass Filter**

This type allows the UIM342AB performing digital low-pass filtering on the input and before collects the level changes. This can eliminate jitters and prevent interference from digital inputs.

Figure 4-2: Low-pass filter trigger



To use this mode, set “TG[i] = T” (i=0, 1 or 2). This instruction sets the filter time T (valid range is 1...60000 ms) of Input i.

• One-Time Shoot

In this mode, UIM342 will processing a port level change only once. After that, the action attached to the input port will be cleared and reset to none. User must set $IL[i]$ again, in order to use the input logic.

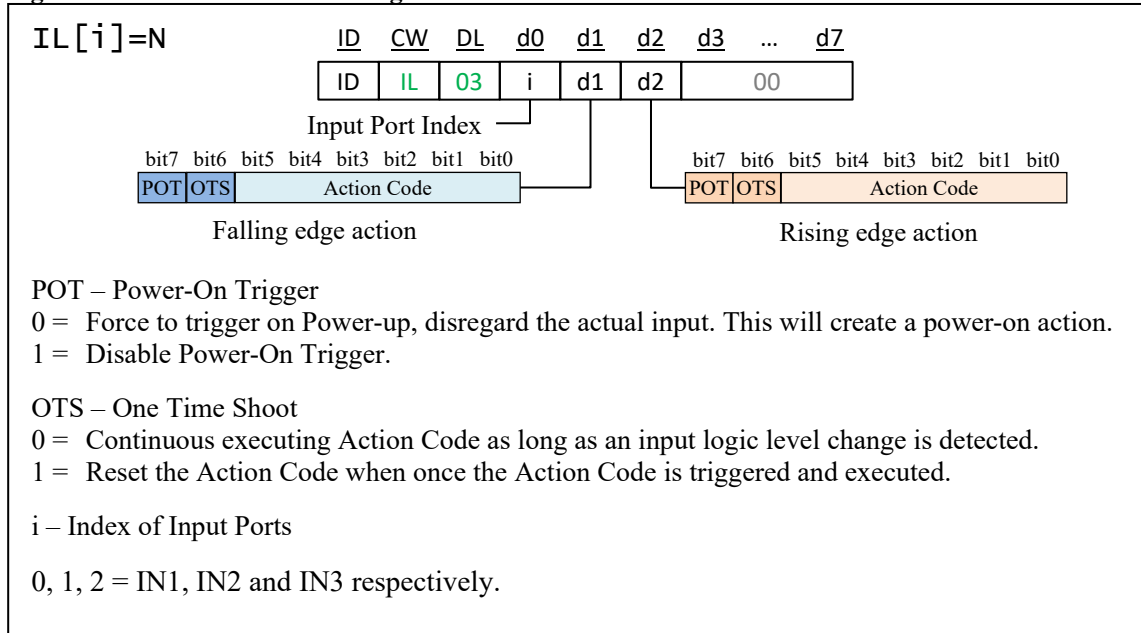
To use this mode, please refer to the following section “Configure Input Logic Action”, and set the OTS bit to 1.

This trigger mode is useful in finding/setting the position origin, preventing the oscillation caused by repeated detection.

4.3 Configure Input Logic Action

Action attached to the input port i is configured by the instruction $IL[i]$ ($i=0, 1, 2$), which is defined as follows:

Figure 4-3: Instruction to attach Logic Action



Action Codes are listed below, that can be attached to the rising/falling edge of the inputs:

Action Code	Action Description	Illustration
0x00	No action	n/a
0x01	Turn the Motor Driver off, freewheel	
0x02	Emergency stop (using SD)	
0x03	Decelerating to stop (using DC)	

Motion Controller with CAN Interface

0x04	Set origin then go reversed relative position (using PR , SP, AC, DC)	
0x05	Set origin then Emergency stop (using SD)	
0x06	Set origin then decelerate to stop (using DC)	
0x07	Reversed Jogging (using SP , AC, DC)	
0x08	Jog (using SP, AC, DC)	
0x09	Go reverse relative position (using PR , SP, AC, DC)	
0x0A	Go relative position (using PR, SP, AC, DC)	
0x0B	Go absolute position (using PA, SP, AC, DC)	
0x0D	Turn off Motor Driver, then Lock Down the Motor Controller (can inquiry but cannot set motion or output)	
0x0E	Emergency stop, then Lock Down the Motor Controller (can inquiry but cannot set motion or output)	

4.4 Real-time Notification for Input Change

When the logic level change on a specific input port is detected, there will be a real-time notification message sent to user. That message could be enabled or disabled by setting “IE[i]=1” or “IE[i]=0” respectively. Here, i=0, 1, 2 represents IN1, IN2, IN3 respectively.

5.0 Instruction Set

This chapter provides details of instructions supported by UIM342.

Note:

- 1) Unless otherwise specified, all message bytes are in hex format.
- 2) Examples assume that the UIM342 address is 5 (ID=5).
- 3) Abbreviation definitions as below:
 - INS – Instruction, from Host (user) to UIM342.
 - ACK – Acknowledgment / Reply, from UIM342 to Host (user).

6.1 PP[i] Protocol Parameter

Protocol Parameter

CW	0x01 (No ACK); 0x81 (Need ACK)													
Data Format	i (unsigned 8-bit); Data (unsigned 8-bit)													
Description	<table><tr><td>PP[i]</td><td colspan="2">Get Protocol Parameters</td></tr><tr><td></td><td>INS data length: 1</td><td>Data: d0 (=i)</td></tr><tr><td></td><td>ACK data length: 2</td><td>Data: d0 (=i), d1</td></tr></table>			PP[i]	Get Protocol Parameters			INS data length: 1	Data: d0 (=i)		ACK data length: 2	Data: d0 (=i), d1		
	PP[i]	Get Protocol Parameters												
		INS data length: 1	Data: d0 (=i)											
		ACK data length: 2	Data: d0 (=i), d1											
	<table><tr><td>PP[i]=N</td><td colspan="2">Set Protocol Parameters</td></tr><tr><td></td><td>INS data length: 2</td><td>Data: d0 (=i), d1</td></tr><tr><td></td><td>ACK data length: 2</td><td>Data: d0 (=i), d1</td></tr></table>			PP[i]=N	Set Protocol Parameters			INS data length: 2	Data: d0 (=i), d1		ACK data length: 2	Data: d0 (=i), d1		
	PP[i]=N	Set Protocol Parameters												
	INS data length: 2	Data: d0 (=i), d1												
	ACK data length: 2	Data: d0 (=i), d1												
<table><tr><td>i</td><td>Description</td><td>Value (N)</td></tr><tr><td>5</td><td>CAN bit rate (bps)</td><td>0: 1000K / 1: 800K / 2: 500K / 3: 250K / 4: 125K</td></tr><tr><td>7</td><td>Node ID</td><td>5...126</td></tr><tr><td>8</td><td>Group ID</td><td>5...126</td></tr></table>			i	Description	Value (N)	5	CAN bit rate (bps)	0: 1000K / 1: 800K / 2: 500K / 3: 250K / 4: 125K	7	Node ID	5...126	8	Group ID	5...126
i	Description	Value (N)												
5	CAN bit rate (bps)	0: 1000K / 1: 800K / 2: 500K / 3: 250K / 4: 125K												
7	Node ID	5...126												
8	Group ID	5...126												
Example GET	○ <u>PP[5]; Get CAN Bitrate.</u>													
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 81 01 05 00 00 00 00 00 00 00 Index [d0] = 0x05. (PP[5])													
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 01 02 05 03 00 00 00 00 00 00 Data [d1] = 0x03. (PP[5] = 3, CAN Bitrate = 250K)													
	○ <u>PP[7]; Get Node ID.</u>													
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 81 01 07 00 00 00 00 00 00 00 Index [d0] = 0x07. (PP[7])													
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 01 02 07 05 00 00 00 00 00 00 Data [d1] = 0x07; (PP[7] = 5, Node ID = 5).													
○ <u>PP[8]; Get Group ID.</u>														
INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 81 01 08 00 00 00 00 00 00 00 Index [d0] = 0x08. (PP[8])														
ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 01 02 08 0A 00 00 00 00 00 00 Data [d1] = 0x0A; (PP[8] = 10, Group ID = 10).														

UIM342 / UIM342S / UIM342XS

Example SET	⊙ <u>PP[5]=2; Set CAN Bitrate = 500K.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 81 02 05 02 00 00 00 00 00 00 <i>Index [d0] = 0x05. (PP[5])</i> <i>Data [d1] = 0x02. (PP[5] = 2, i.e., 500K)</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 01 02 05 02 00 00 00 00 00 00 ⊙ <u>PP[7]=12; Set Module / Motor Node ID = 12.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 81 02 07 0B 00 00 00 00 00 00 <i>Index [d0] = 0x07. (PP[7])</i> <i>Data [d1] = 0x0B = 12. (PP[7] = 12)</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 01 02 07 0B 00 00 00 00 00 00 ⊙ <u>PP[8]=20; Set Module / Motor Group ID = 20.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 81 02 08 14 00 00 00 00 00 00 <i>Index [d0] = 0x08. (PP[8])</i> <i>Data [d1] = 0x14 = 20. (PP[8] = 20)</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 01 02 08 14 00 00 00 00 00 00
Note	– Within a specific CAN network, Node IDs and Group IDs of all UIM devices should never be overlapped. – Protocol Parameters will take effectiveness after reboot the UIM device. – The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off.

Motion Controller with CAN Interface

6.2 IC[i] Initial Configuration

System Settings

CW	0x06 (No ACK); 0x86 (Need ACK)																																											
Data Format	i (unsigned 8-bit); Data (unsigned 16-bit)																																											
Description	<table><tr><td>IC[i]</td><td colspan="3">Get initial state configuration of UIM342 when Power-On</td></tr><tr><td></td><td>INS data length: 1</td><td colspan="2">Data: d0 (=i)</td></tr><tr><td></td><td>ACK data length: 3</td><td colspan="2">Data: d0 (=i), d1, d2</td></tr></table>				IC[i]	Get initial state configuration of UIM342 when Power-On				INS data length: 1	Data: d0 (=i)			ACK data length: 3	Data: d0 (=i), d1, d2																													
	IC[i]	Get initial state configuration of UIM342 when Power-On																																										
		INS data length: 1	Data: d0 (=i)																																									
		ACK data length: 3	Data: d0 (=i), d1, d2																																									
	<table><tr><td>IC[i]=N</td><td colspan="3">Set initial state configuration of UIM342 when Power-On</td></tr><tr><td></td><td>INS data length: 3</td><td colspan="2">Data: d0 (=i), d1, d2</td></tr><tr><td></td><td>ACK data length: 3</td><td colspan="2">Data: d0 (=i), d1, d2</td></tr></table>				IC[i]=N	Set initial state configuration of UIM342 when Power-On				INS data length: 3	Data: d0 (=i), d1, d2			ACK data length: 3	Data: d0 (=i), d1, d2																													
	IC[i]=N	Set initial state configuration of UIM342 when Power-On																																										
		INS data length: 3	Data: d0 (=i), d1, d2																																									
		ACK data length: 3	Data: d0 (=i), d1, d2																																									
	<table><tr><td>i</td><td>Description</td><td colspan="2">Value (N)</td></tr><tr><td>0</td><td>Auto-Enable Motor Driver after Power up</td><td>0: Disable</td><td>1: Enable</td></tr><tr><td>1</td><td>Positive Motor Direction</td><td>0: CW</td><td>1: CCW</td></tr><tr><td>3</td><td>Enable Lockdown System</td><td>0: Disable</td><td>1: Enable</td></tr><tr><td>4</td><td>Units for AC and DC</td><td>0: pulse/sec²</td><td>1: millisecond</td></tr><tr><td>6</td><td>Using Closed-loop Control</td><td>0: Open loop</td><td>1: Closed loop</td></tr><tr><td>7</td><td>Software Limit</td><td>0: Disable</td><td>1: Enable</td></tr><tr><td>8</td><td>Internally Controlled Brake Logic⁽¹⁾</td><td>0: Disable</td><td>1: Enable</td></tr><tr><td>12</td><td>Auto-Release Brake after Power-up</td><td>0: Disable</td><td>1: Enable</td></tr><tr><td>15</td><td>Using Internally Controlled Brake⁽²⁾</td><td>0: Disable</td><td>1: Enable</td></tr></table>				i	Description	Value (N)		0	Auto-Enable Motor Driver after Power up	0: Disable	1: Enable	1	Positive Motor Direction	0: CW	1: CCW	3	Enable Lockdown System	0: Disable	1: Enable	4	Units for AC and DC	0: pulse/sec ²	1: millisecond	6	Using Closed-loop Control	0: Open loop	1: Closed loop	7	Software Limit	0: Disable	1: Enable	8	Internally Controlled Brake Logic ⁽¹⁾	0: Disable	1: Enable	12	Auto-Release Brake after Power-up	0: Disable	1: Enable	15	Using Internally Controlled Brake ⁽²⁾	0: Disable	1: Enable
	i	Description	Value (N)																																									
	0	Auto-Enable Motor Driver after Power up	0: Disable	1: Enable																																								
	1	Positive Motor Direction	0: CW	1: CCW																																								
	3	Enable Lockdown System	0: Disable	1: Enable																																								
	4	Units for AC and DC	0: pulse/sec ²	1: millisecond																																								
	6	Using Closed-loop Control	0: Open loop	1: Closed loop																																								
7	Software Limit	0: Disable	1: Enable																																									
8	Internally Controlled Brake Logic ⁽¹⁾	0: Disable	1: Enable																																									
12	Auto-Release Brake after Power-up	0: Disable	1: Enable																																									
15	Using Internally Controlled Brake ⁽²⁾	0: Disable	1: Enable																																									
Remark:																																												
(1) For Motor with Internally Controlled Brake only. If enabled, the motor driver will not be able to turn off until the Brake is released (MT[5]=0;) to prevent slipping; meanwhile, the BG instruction will be ignored when the Brake is engaged (Lockdown) to prevent damage to the braking disk.																																												
(2) This bit must be enabled if the motor is equipped with the internally controlled Brake. Otherwise, the Brake cannot be controlled by software.																																												
Example GET	○ IC[0]; Get the Setting of “Auto-Enable Motor Driver after Power up”.																																											
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 01 00 00 00 00 00 00 00 Index [d0] = 0x00. (IC[0]).																																											
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 00 00 00 00 00 00 00 Data [d2: d1] = 0x0000; (IC[0] = Disabled).																																											
	○ IC[1]; Get the Setting of “Positive Motor Direction” .																																											
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 01 01 00 00 00 00 00 00 Index [d0] = 0x01.(IC[1]).																																											
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 01 01 00 00 00 00 00 Data [d2: d1] = 0x0001. (IC[1] = 1, CCW).																																											

Example GET	○ <u>IC[3]; Get the Setting of “Enable Lockdown System”</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 01 03 00 00 00 00 00 00 00 <i>Index [d0] = 0x03. (IC[3]).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 03 01 00 00 00 00 00 00 <i>Data [d2: d1] = 0x0001. (IC[3] = 1, Motor System will be Locked Down if input INx is triggered, which makes it working like an Emergency Button).</i> ○ <u>IC[4]; Get the Setting of “Units for AC and DC”</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 01 04 00 00 00 00 00 00 00 <i>Index [d0] = 0x04. (IC[4]).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 04 01 00 00 00 00 00 00 <i>Data [d2: d1] = 0x0001. (IC[4] = 1, Acceleration / Deceleration Unit is Millisecond).</i> ○ <u>IC[6]; Get the Setting of “Using Closed-loop Control”</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 01 06 00 00 00 00 00 00 00 <i>Index [d0] = 0x06. (IC[6]).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 06 01 00 00 00 00 00 00 <i>Data [d2: d1] = 0x0001. (IC[6] = 1, Using Closed-loop Control mode).</i> ○ <u>IC[7]; Get the Setting of “Enable Software Limit”</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 01 07 00 00 00 00 00 00 00 <i>Index [d0] = 0x07. (IC[7]).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 07 01 00 00 00 00 00 00 <i>Data [d2: d1] = 0x0001. (IC[7] = 1, Software Limit is Enabled).</i> ○ <u>IC[8]; Get the Setting of “ Internally Controlled Brake Logic ”</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 01 08 00 00 00 00 00 00 00 <i>Index [d0] = 0x08; (IC[8])</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 08 01 00 00 00 00 00 00 <i>Data [d2: d1] = 0x0001; (IC[8] = 1, Internally Controlled Brake Logic is Enabled).</i>
------------------------	--

Motion Controller with CAN Interface

Example GET	○ <u>IC[15]; Get the Setting of “ Using Internally Controlled Brake ”</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 01 0F 00 00 00 00 00 00 00 <i>Index [d0] = 0x0F; (IC[15])</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 0F 01 00 00 00 00 00 00 <i>Data [d2: d1] = 0x0001; (IC[15] = 1, Internally Controlled Brake is used).</i>
Example SET	● <u>IC[0]=0; Disable “Auto-Enable Motor Driver after Power up”</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 03 00 00 00 00 00 00 00 00 <i>Index [d0] = 0x00.</i> <i>Data [d2: d1] = 0x0000; (IC[0] = Disable).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 00 00 00 00 00 00 00 00 ● <u>IC[1]=1; Set “Positive Motor Direction = CCW”</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 03 01 01 00 00 00 00 00 00 <i>Index [d0] = 0x01.</i> <i>Data [d2: d1] = 0x0001. (IC[1] = 1, CCW).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 01 01 00 00 00 00 00 00 ● <u>IC[3]=0; Disable “Lockdown System”</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 03 03 00 00 00 00 00 00 00 <i>Index [d0] = 0x03.</i> <i>Data [d2: d1] = 0x0000.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 03 00 00 00 00 00 00 00 ● <u>IC[4]=1; Set the units for AC and DC to millisecond</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 03 04 01 00 00 00 00 00 00 <i>Index [d0] = 0x04. (IC[4])</i> <i>Data [d2:d1] = 0x0001. (IC[4] = 1).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 04 01 00 00 00 00 00 00

Example SET	⊙ <u>IC[6]=1; Using Closed-loop Control Mode</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 03 06 01 00 00 00 00 00 <i>Index [d0] = 0x06; (IC[6])</i> <i>Data [d2:d1] = 0x0001; (IC[6] = Using Closed-loop Control).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 06 01 00 00 00 00 00 ⊙ <u>IC[7]=1; Enable Software Limits</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 03 07 01 00 00 00 00 00 <i>Index [d0] = 0x07; (IC[7])</i> <i>Data [d2:d1] = 0x0001; (IC[7] = 1; Enable Software Limits).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 07 01 00 00 00 00 00 ⊙ <u>IC[8]=1; Enable Internally Controlled Brake Logic</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 03 08 01 00 00 00 00 00 <i>Index [d0] = 0x08; (IC[8])</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 08 01 00 00 00 00 00 <i>Data [d2: d1] = 0x0001; (IC[8] = 1, Enable Internally Controlled Brake Logic).</i> ⊙ <u>IC[15]=1; Using Internally Controlled Brake</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 86 03 0F 01 00 00 00 00 00 <i>Index [d0] = 0x0F; (IC[15])</i> <i>Data [d2: d1] = 0x0001; (IC[15] = 1, Using Internally Controlled Brake).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 06 03 0F 01 00 00 00 00 00
Note	– Within a specific CAN network, Node IDs and Group IDs of all UIM devices should never be overlapped. – Protocol Parameters will take effectiveness after reboot the UIM device. – The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off.

6.3 IE[i] Inform Enable

System Settings

CW	0x07 (No ACK); 0x87 (Need ACK)																															
Data Format	i (unsigned 8-bit); Data (unsigned 16-bit)																															
Description	<table><tr><td>IE[i]</td><td colspan="3">Get Inform Enable Configuration</td></tr><tr><td></td><td>INS data length: 1</td><td colspan="2">Data: d0 (=i)</td></tr><tr><td></td><td>ACK data length: 3</td><td colspan="2">Data: d0 (=i), d1, d2</td></tr></table>				IE[i]	Get Inform Enable Configuration				INS data length: 1	Data: d0 (=i)			ACK data length: 3	Data: d0 (=i), d1, d2																	
	IE[i]	Get Inform Enable Configuration																														
		INS data length: 1	Data: d0 (=i)																													
		ACK data length: 3	Data: d0 (=i), d1, d2																													
	<table><tr><td>IE[i]=N</td><td colspan="3">Set Inform Enable Configuration</td></tr><tr><td></td><td>INS data length: 3</td><td colspan="2">Data: d0 (=i), d1, d2</td></tr><tr><td></td><td>ACK data length: 3</td><td colspan="2">Data: d0 (=i), d1, d2</td></tr></table>				IE[i]=N	Set Inform Enable Configuration				INS data length: 3	Data: d0 (=i), d1, d2			ACK data length: 3	Data: d0 (=i), d1, d2																	
	IE[i]=N	Set Inform Enable Configuration																														
		INS data length: 3	Data: d0 (=i), d1, d2																													
		ACK data length: 3	Data: d0 (=i), d1, d2																													
	<table><tr><td>i</td><td>Description</td><td colspan="2">Value (N)</td></tr><tr><td>0</td><td>Port IN1 change notification</td><td>0: disable</td><td>1: enable</td></tr><tr><td>1</td><td>Port IN2 change notification</td><td>0: disable</td><td>1: enable</td></tr><tr><td>2</td><td>Port IN3 change notification</td><td>0: disable</td><td>1: enable</td></tr><tr><td>8</td><td>PTP positioning finish notification</td><td>0: disable</td><td>1: enable</td></tr><tr><td>10</td><td>PVT/PT FIFO Empty (dry out) Notification</td><td>0: disable</td><td>1: enable</td></tr><tr><td>11</td><td>PVT/PT FIFO Low Warning Notification</td><td>0: disable</td><td>1: enable</td></tr></table>				i	Description	Value (N)		0	Port IN1 change notification	0: disable	1: enable	1	Port IN2 change notification	0: disable	1: enable	2	Port IN3 change notification	0: disable	1: enable	8	PTP positioning finish notification	0: disable	1: enable	10	PVT/PT FIFO Empty (dry out) Notification	0: disable	1: enable	11	PVT/PT FIFO Low Warning Notification	0: disable	1: enable
	i	Description	Value (N)																													
0	Port IN1 change notification	0: disable	1: enable																													
1	Port IN2 change notification	0: disable	1: enable																													
2	Port IN3 change notification	0: disable	1: enable																													
8	PTP positioning finish notification	0: disable	1: enable																													
10	PVT/PT FIFO Empty (dry out) Notification	0: disable	1: enable																													
11	PVT/PT FIFO Low Warning Notification	0: disable	1: enable																													
Example GET	○ <u>IE[0]; Get IN1's IE configure.</u>																															
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 87 01 00 00 00 00 00 00 00 Index [d0] = 0x00. (IN1's IE configure) ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 07 03 00 01 00 00 00 00 00 Index [d0] = 0x00; (IN1's IE configure) Data [d2:d1] = 0x0001. (Enabled)																															
Example SET	● <u>IE[1]=1; Set Enable IN2's change notification.</u>																															
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 87 03 01 01 00 00 00 00 00 Index [d0] = 0x01. (IN2's change notification) Data [d2:d1] = 0x0001. (Set Enable) ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 07 03 01 01 00 00 00 00 00																															
Note	– Set/Get the information enable configuration. – To receive the real-time change notification, the corresponding information enables bit must be set (=1). – Refer to chapter 2.4 for more details about real-time change notification. – The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off.																															

6.4 ML Model

System Settings

CW	0x8B (Need ACK)																																																																																								
Data Format	Segment Data																																																																																								
Description	<table><tr><td>ML</td><td colspan="5">Get the model, function module and firmware version</td></tr><tr><td></td><td colspan="3">INS data length: 0</td><td colspan="2">Data: n/a</td></tr><tr><td></td><td colspan="3">ACK data length: 8</td><td colspan="2">Data: d0... d7</td></tr></table>						ML	Get the model, function module and firmware version						INS data length: 0			Data: n/a			ACK data length: 8			Data: d0... d7																																																																		
	ML	Get the model, function module and firmware version																																																																																							
		INS data length: 0			Data: n/a																																																																																				
		ACK data length: 8			Data: d0... d7																																																																																				
○ <u>ML; Get Model.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 8B 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 0B 08 20 0A 01 00 00 00 00 For UIM342CM series, please refer to the following table for model info. <table><tr><td>Model</td><td>d0</td><td>d1</td><td>d2</td><td>d3</td><td>d5:d4</td><td>d7:d6</td></tr><tr><td>342XS</td><td>0x20</td><td>0x0A</td><td>0x01</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr><tr><td>342S</td><td>0x21</td><td>0x0A</td><td>0x01</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr><tr><td>342H02</td><td>0x22</td><td>0x14</td><td>0x09</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr><tr><td>342H04</td><td>0x22</td><td>0x28</td><td>0x09</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr><tr><td>342H08</td><td>0x22</td><td>0x50</td><td>0x09</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr></table> xxxx – Factory use, don’t care. Example GET For UIM342CA series, please refer to the following table for model info. <table><tr><td>Model</td><td>d0</td><td>d1</td><td>d2</td><td>d3</td><td>d5:d4</td><td>d7:d6</td></tr><tr><td>342XSCA</td><td>0x20</td><td>0x0A</td><td>0x02</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr><tr><td>342SCA</td><td>0x21</td><td>0x0A</td><td>0x02</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr><tr><td>342H02CA</td><td>0x22</td><td>0x14</td><td>0x0A</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr><tr><td>342H04CA</td><td>0x22</td><td>0x28</td><td>0x0A</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr><tr><td>342H08CA</td><td>0x22</td><td>0x50</td><td>0x0A</td><td>xx</td><td>Firmware version</td><td>xxxx</td></tr></table> xxxx – Factory use, don’t care.						Model	d0	d1	d2	d3	d5:d4	d7:d6	342XS	0x20	0x0A	0x01	xx	Firmware version	xxxx	342S	0x21	0x0A	0x01	xx	Firmware version	xxxx	342H02	0x22	0x14	0x09	xx	Firmware version	xxxx	342H04	0x22	0x28	0x09	xx	Firmware version	xxxx	342H08	0x22	0x50	0x09	xx	Firmware version	xxxx	Model	d0	d1	d2	d3	d5:d4	d7:d6	342XSCA	0x20	0x0A	0x02	xx	Firmware version	xxxx	342SCA	0x21	0x0A	0x02	xx	Firmware version	xxxx	342H02CA	0x22	0x14	0x0A	xx	Firmware version	xxxx	342H04CA	0x22	0x28	0x0A	xx	Firmware version	xxxx	342H08CA	0x22	0x50	0x0A	xx	Firmware version	xxxx
Model	d0	d1	d2	d3	d5:d4	d7:d6																																																																																			
342XS	0x20	0x0A	0x01	xx	Firmware version	xxxx																																																																																			
342S	0x21	0x0A	0x01	xx	Firmware version	xxxx																																																																																			
342H02	0x22	0x14	0x09	xx	Firmware version	xxxx																																																																																			
342H04	0x22	0x28	0x09	xx	Firmware version	xxxx																																																																																			
342H08	0x22	0x50	0x09	xx	Firmware version	xxxx																																																																																			
Model	d0	d1	d2	d3	d5:d4	d7:d6																																																																																			
342XSCA	0x20	0x0A	0x02	xx	Firmware version	xxxx																																																																																			
342SCA	0x21	0x0A	0x02	xx	Firmware version	xxxx																																																																																			
342H02CA	0x22	0x14	0x0A	xx	Firmware version	xxxx																																																																																			
342H04CA	0x22	0x28	0x0A	xx	Firmware version	xxxx																																																																																			
342H08CA	0x22	0x50	0x0A	xx	Firmware version	xxxx																																																																																			

6.5 SN Serial Number

System Settings

CW	0x8C (Need ACK)		
Data Format	Data (unsigned 32-bit)		
Description			
	SN	Get device's Serial Number	
		INS data length: 0	Data: n/a
		ACK data length: 4	Data: d0...d3
Example GET			
	○ <u>SN; Get Serial Number.</u>		
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 8C 00 00 00 00 00 00 00 00		
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 0C 08 01 02 03 04 00 00 00 [d3:d2:d1:d0] = 0x04030201 (Serial number 0067305985).		

6.6 ER[i] Error Report

System Settings

CW	0x0F (No ACK); 0x8F (Need ACK)											
Data Format	i (unsigned 8-bit); Data (unsigned 8-bit)											
Description	<table><tr><td>ER[i]</td><td colspan="2">Get Error Information</td></tr><tr><td></td><td>INS data length: 1</td><td>Data: d0 (=i)</td></tr><tr><td></td><td>ACK data length: 6</td><td>Data: d0 (=i), d1, d2, d3, d4, d5</td></tr></table>			ER[i]	Get Error Information			INS data length: 1	Data: d0 (=i)		ACK data length: 6	Data: d0 (=i), d1, d2, d3, d4, d5
	ER[i]	Get Error Information										
		INS data length: 1	Data: d0 (=i)									
		ACK data length: 6	Data: d0 (=i), d1, d2, d3, d4, d5									
	<table><tr><td>ER[0]=0</td><td colspan="2">Clear All Error Info.</td></tr><tr><td></td><td>INS data length: 2</td><td>Data: d0 (=i), d1, d2</td></tr><tr><td></td><td>ACK data length: 6</td><td>Data: d0 (=i), d1, d2, d3, d4, d5</td></tr></table>			ER[0]=0	Clear All Error Info.			INS data length: 2	Data: d0 (=i), d1, d2		ACK data length: 6	Data: d0 (=i), d1, d2, d3, d4, d5
	ER[0]=0	Clear All Error Info.										
		INS data length: 2	Data: d0 (=i), d1, d2									
	ACK data length: 6	Data: d0 (=i), d1, d2, d3, d4, d5										
<table><tr><td>i</td><td>Description</td></tr><tr><td>0</td><td>Get the Newest Error Info. / Reset All Error Info.</td></tr><tr><td>10...18</td><td>Get the Historic 1st...9th (Newest...Oldest) Error Info.</td></tr></table>			i	Description	0	Get the Newest Error Info. / Reset All Error Info.	10...18	Get the Historic 1 st ...9 th (Newest...Oldest) Error Info.				
i	Description											
0	Get the Newest Error Info. / Reset All Error Info.											
10...18	Get the Historic 1 st ...9 th (Newest...Oldest) Error Info.											
Refer to chapter 2.3 “ Error Report ” for the description of d0...d5.												
Example GET	○ <u>ER[0]; Get the Newest Error Code.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 8F 01 00 00 00 00 00 00 00 <i>Index [d0] = 0x00.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 0F 06 00 33 81 05 00 00 00 <i>Index [d0] = 0x00;</i> <i>Data [d1] = 0x33; (Error code = 0x33, i.e. instruction data error).</i> <i>Data [d2] = 0x81; (CW related to the error = 0x81, i.e. PP).</i> <i>Data [d3] = 0x05; (Sub-Index of the CW related to the error = 5).</i> <i>Data [d5:d4] = 0x00; (Reserve).</i> <i>The newest error = "In the instruction PP[5]=N, the data is illegal".</i>											
Example SET	⊙ <u>ER[0] = 0; Reset All Error.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 8F 02 00 00 00 00 00 00 00 <i>Index [d0] = 0x00; (All Error).</i> <i>Data [d1] = 0x00; (Reset Error).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 0F 06 00 00 00 00 00 00 00 <i>Index [d0] = 0x00; (All Error).</i> <i>Data [d1]... [d5] = 0; (No Error).</i>											

6.7 QE[i] Quadrature Encoder

System Settings

CW	0x3D (No ACK); 0xBD (Need ACK)																
Data Format	i (unsigned 8-bit); Data (unsigned 16-bit)																
Description	<table><tr><td>QE[i]</td><td colspan="2">Get Encoder Parameters</td></tr><tr><td></td><td>INS data length: 1</td><td>Data: d0 (=i)</td></tr><tr><td></td><td>ACK data length: 3</td><td>Data: d0 (=i), d1, d2</td></tr></table>			QE[i]	Get Encoder Parameters			INS data length: 1	Data: d0 (=i)		ACK data length: 3	Data: d0 (=i), d1, d2					
	QE[i]	Get Encoder Parameters															
		INS data length: 1	Data: d0 (=i)														
		ACK data length: 3	Data: d0 (=i), d1, d2														
	<table><tr><td>QE[i]=N</td><td colspan="2">Set Encoder Parameters</td></tr><tr><td></td><td>INS data length: 3</td><td>Data: d0 (=i), d1, d2</td></tr><tr><td></td><td>ACK data length: 3</td><td>Data: d0 (=i), d1, d2</td></tr></table>			QE[i]=N	Set Encoder Parameters			INS data length: 3	Data: d0 (=i), d1, d2		ACK data length: 3	Data: d0 (=i), d1, d2					
	QE[i]=N	Set Encoder Parameters															
		INS data length: 3	Data: d0 (=i), d1, d2														
		ACK data length: 3	Data: d0 (=i), d1, d2														
<table><tr><td>i</td><td>Description</td><td>Value (N)</td></tr><tr><td>0</td><td>Lines per Revolution of encoder (LPR)</td><td>1..65535</td></tr><tr><td>1</td><td>Max Position (Pulse) Error. (Exceeding this value, the stalling process starts.)</td><td>10..65535 Pulse</td></tr><tr><td>3</td><td>Battery Voltage of the Absolute Encoder</td><td>0...3300 (0.000...3.300V)</td></tr><tr><td>4</td><td>Counts per revolution (CPR = Micro Steps*200)</td><td>200..25600 (1...128*200)</td></tr></table>			i	Description	Value (N)	0	Lines per Revolution of encoder (LPR)	1..65535	1	Max Position (Pulse) Error. (Exceeding this value, the stalling process starts.)	10..65535 Pulse	3	Battery Voltage of the Absolute Encoder	0...3300 (0.000...3.300V)	4	Counts per revolution (CPR = Micro Steps*200)	200..25600 (1...128*200)
i	Description	Value (N)															
0	Lines per Revolution of encoder (LPR)	1..65535															
1	Max Position (Pulse) Error. (Exceeding this value, the stalling process starts.)	10..65535 Pulse															
3	Battery Voltage of the Absolute Encoder	0...3300 (0.000...3.300V)															
4	Counts per revolution (CPR = Micro Steps*200)	200..25600 (1...128*200)															
Example GET	○ QE[0]; Get Counts per Revolution for closed-loop control. INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 BD 01 00 00 00 00 00 00 00 Index [d0] = 0x04. (Lines per Revolution). ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 3D 03 00 00 10 00 00 00 00 Data [d2: d1] = 0x0100 = 4096. (Lines per Revolution=4096).																
	○ QE[1]; Get Max Position (Pulse) Error. INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 BD 01 01 00 00 00 00 00 00 Index [d0] = 0x01. (Max Position Error, Missing Steps). ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 3D 03 01 E8 03 00 00 00 00 Data [d2: d1] = 0x03E8 = 1000. (Max Position Error = 1000).																
	○ QE[3]; Get Absolute Encoder Battery Voltage. INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 BD 01 03 00 00 00 00 00 00 Index [d0] = 0x03.(Absolute Encoder Battery Voltage) ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 3D 03 03 54 08 00 00 00 00 Data [d2: d1] = 0x0854 = 2900. (2900 = 2.900V).																

Example GET	○ <u>QE[4]; Get Counts per Revolution.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 BD 01 04 00 00 00 00 00 00 00 <i>Index [d0] = 0x04.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 3D 03 04 80 0C 00 00 00 00 00 <i>Data [d2:d1] = 0x0C80. (Counts per Revolution = 3200).</i>
Example SET	● <u>QE[1] = 500; Set Max Position (Pulse) Error.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 BD 03 01 F4 01 00 00 00 00 00 <i>Index [d0] = 0x01. (Max Position Error).</i> <i>Data [d2: d1] = 0x01F4 = 500. (Maximum Allowed Missing Steps=500).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 3D 03 01 F4 01 00 00 00 00 00
Note	– QE[0] (LPR) is a physical property of the encoder. Setting it incorrectly can cause unpredictable behavior. – Adjust QE[4] (CPR) by changing the Micro-stepping Resolution (MT[0]). After adjustment, power off and restart the system for changes to take effect. – The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off.

6.8 SY[i] System Operation

System Settings

CW	0x7E (No ACK)											
Data Format	i (unsigned 8-bit)											
Description	<table><tr><td>SY[i]</td><td colspan="2">System Operation</td></tr><tr><td></td><td>INS data length: 1</td><td>Data: d0 (=i)</td></tr><tr><td></td><td colspan="2">No ACK</td></tr></table>			SY[i]	System Operation			INS data length: 1	Data: d0 (=i)		No ACK	
	SY[i]	System Operation										
		INS data length: 1	Data: d0 (=i)									
		No ACK										
	<table><tr><td>i</td><td>Description</td></tr><tr><td>2</td><td>Restore factory default settings.</td></tr></table>			i	Description	2	Restore factory default settings.					
i	Description											
2	Restore factory default settings.											
Example SET	⦿ <u>SY[2]; Restore factory defaults.</u>											
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 7E 01 02 00 00 00 00 00 00 00 00 <i>Index [d0] = 0x02;</i>											

6.9 MT[i] Motor Driver

Motor Driver

CW	0x10 (No ACK); 0x90 (Need ACK)																										
Data Format	i (unsigned 8-bit); Data (unsigned 16-bit)																										
Description	<table><tr><td>MT[i]</td><td colspan="2">Get Motor Driver Parameters</td></tr><tr><td></td><td>INS data length: 1</td><td>Data: d0 (=i)</td></tr><tr><td></td><td>ACK data length: 3</td><td>Data: d0 (=i), d1, d2</td></tr></table>			MT[i]	Get Motor Driver Parameters			INS data length: 1	Data: d0 (=i)		ACK data length: 3	Data: d0 (=i), d1, d2															
	MT[i]	Get Motor Driver Parameters																									
		INS data length: 1	Data: d0 (=i)																								
		ACK data length: 3	Data: d0 (=i), d1, d2																								
	<table><tr><td>MT[i]=N</td><td colspan="2">Set Motor Driver Parameters</td></tr><tr><td></td><td>INS data length: 3</td><td>Data: d0 (=i), d1, d2</td></tr><tr><td></td><td>ACK data length: 3</td><td>Data: d0 (=i), d1, d2</td></tr></table>			MT[i]=N	Set Motor Driver Parameters			INS data length: 3	Data: d0 (=i), d1, d2		ACK data length: 3	Data: d0 (=i), d1, d2															
	MT[i]=N	Set Motor Driver Parameters																									
		INS data length: 3	Data: d0 (=i), d1, d2																								
		ACK data length: 3	Data: d0 (=i), d1, d2																								
	<table><tr><td>i</td><td>Description</td><td colspan="2">Value (N)</td></tr><tr><td>0</td><td>Micro-Stepping Resolution</td><td colspan="2">1/2/4/8/16/32/64/128</td></tr><tr><td>1</td><td>Working Current</td><td colspan="2">5 ... 80 (= 0.5 ... 8.0 Amp)</td></tr><tr><td>2</td><td>% of Idle Current over Working Current</td><td colspan="2">0...100 (= 0...100%)</td></tr><tr><td>3</td><td>Delay of Automatic Enable after Power-on</td><td colspan="2">0...60000 (millisecond)</td></tr><tr><td>5</td><td>Release/Engage Internally Controlled Brake⁽¹⁾</td><td>0: Release</td><td>1: Engage</td></tr></table>			i	Description	Value (N)		0	Micro-Stepping Resolution	1/2/4/8/16/32/64/128		1	Working Current	5 ... 80 (= 0.5 ... 8.0 Amp)		2	% of Idle Current over Working Current	0...100 (= 0...100%)		3	Delay of Automatic Enable after Power-on	0...60000 (millisecond)		5	Release/Engage Internally Controlled Brake ⁽¹⁾	0: Release	1: Engage
	i	Description	Value (N)																								
0	Micro-Stepping Resolution	1/2/4/8/16/32/64/128																									
1	Working Current	5 ... 80 (= 0.5 ... 8.0 Amp)																									
2	% of Idle Current over Working Current	0...100 (= 0...100%)																									
3	Delay of Automatic Enable after Power-on	0...60000 (millisecond)																									
5	Release/Engage Internally Controlled Brake ⁽¹⁾	0: Release	1: Engage																								
Remark:																											
(1) Release brake: powering the brake, the motor shaft will be free;																											
Engage brake: un-powering on the brake, the motor shaft will be locked.																											
Example GET	○ <u>MT[0]; Get Micro-Stepping Resolution.</u>																										
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 01 00 00 00 00 00 00 00 Index [d0] = 0x00. (Micro-Stepping Resolution).																										
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 00 10 00 00 00 00 00 Data [d2: d1] = 0x0010 = 16.																										
	○ <u>MT[1]; Get Working Current.</u>																										
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 01 01 00 00 00 00 00 00 Index [d0] = 0x01. (Working Current).																										
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 01 1C 00 00 00 00 00 Data [d2: d1] = 0x001C = 28. (2.8Amp)																										
	○ <u>MT[2]; Get Idle Current.</u>																										
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 01 02 00 00 00 00 00 00 Index [d0] = 0x02. (Idle Current).																										
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 02 32 00 00 00 00 00 Data [d2: d1] = 0x0032 = 50. (50% of Working Current).																										

Motion Controller with CAN Interface

Example GET	○ <u>MT[3]; Get Delay of Automatic Enable after Power-on.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 01 03 00 00 00 00 00 00 00 <i>Index [d0] = 0x03. (Delay of Automatic Enable after Power-on).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 03 E8 03 00 00 00 00 00 <i>Data [d2: d1] = 0x03E8 = 1000. (Delay = 1000ms).</i> ○ <u>MT[5]; Get Internal Controlled Brake Status.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 01 05 00 00 00 00 00 00 00 <i>Index [d0] = 0x05. (Internal Controlled Brake Status).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 05 01 00 00 00 00 00 00 <i>Data [d2: d1] = 0x0001. (Brake is Locked, prevent motor from move).</i>
Example SET	● <u>MT[0] = 16; Set Micro-stepping resolution to 16.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 03 00 10 00 00 00 00 00 00 <i>Index [d0] = 0x00; (Micro-stepping Resolution).</i> <i>Data [d2: d1] = 0x0010 = 16.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 00 10 00 00 00 00 00 00 ● <u>MT[1] = 28; Set Working Current to 2.8Amp.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 03 01 1C 00 00 00 00 00 00 <i>Index [d0] = 0x01; (Working Current).</i> <i>Data [d2: d1] = 0x001C = 28. (2.8Amp)</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 01 1C 00 00 00 00 00 00 ● <u>MT[2] = 50; Set Idle Current to 50% of Working Current.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 03 02 32 00 00 00 00 00 00 <i>Index [d0] = 0x02; (Idle Current).</i> <i>Data [d2: d1] = 0x0032 = 50. (50%)</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 02 32 00 00 00 00 00 00

Example SET	⊙ <u>MT[3] = 1000; Set Delay of Automatic Enable after Power-on to 1000ms.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 03 03 E8 03 00 00 00 00 00 <i>Index [d0] = 0x03. (Delay of Automatic Enable after Power-on).</i> <i>Data [d2: d1] = 0x03E8 = 1000. (Delay = 1000ms).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 03 E8 03 00 00 00 00 00 ⊙ <u>MT[5] = 1; Set Internal Controlled Brake to Lockdown the motor.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 03 05 01 00 00 00 00 00 00 <i>Index [d0] = 0x05. (Internal Controlled Brake).</i> <i>Data [d2: d1] = 0x0001. (Set Brake to Lockdown the motor).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 05 01 00 00 00 00 00 00 ⊙ <u>MT[5] = 0; Set Internal Controlled Brake to Release the motor.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 90 03 05 00 00 00 00 00 00 00 <i>Index [d0] = 0x05. (Internal Controlled Brake).</i> <i>Data [d2: d1] = 0x0000. (Set Brake to Release).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 10 03 05 00 00 00 00 00 00 00
Note	– The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off. – When setting micro-stepping resolution, counts per revolution for closed-loop control needs to be set simultaneously. After the setting is completed, it needs to be powered off and restarted to take effect.

Motion Controller with CAN Interface

6.10 M0 Motor Driver On /Off

Motor Driver

CW	0x15 (No ACK) / 0x95 (Need ACK)		
Data Format	unsigned 8-bit		
Description	M0	Get Motor Drive ON/OFF status	
		INS data length: 0	Data: n/a
		ACK data length: 1	Data: d0
	M0=N	Set Motor Drive ON (N=1) / OFF (N=0)	
		INS data length: 1	Data: d0
		ACK data length: 1	Data: d0
Example GET	○ <u>M0; Get motor driver on/off status.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 95 00 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 15 01 01 00 00 00 00 00 00 00 <i>Data [d0] = 0x01. (The Motor Driver is ON).</i>		
	● <u>M0=1; Turn the Motor Driver ON.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 95 01 01 00 00 00 00 00 00 00 <i>Data [d0] = 0x01.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 15 01 01 00 00 00 00 00 00 00 ● <u>M0=0; Turn the Motor Driver OFF.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 95 01 00 00 00 00 00 00 00 00 <i>Data [d0] = 0x00.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 15 01 00 00 00 00 00 00 00 00		
Note	– Make sure the motor driver is ON (M0=1), before sending instructions such as SP, PR, JV and BG etc. Otherwise, unexpected movement will happen (caused by previous residual actions) when turning on the motor driver. – The motor can be driven only after the motor driver is ON. – In OFF status, the motor is free-wheeled, but the logic circuit still works.		

6.11 BG Begin Motion

Motion Control

CW	0x16 (NO ACK); 0x96 (Need ACK)											
Data Format	n/a											
Description	<table><tr><td>BG</td><td colspan="2">Begin Motion</td></tr><tr><td></td><td>INS data length: 0</td><td>Data: n/a</td></tr><tr><td></td><td>ACK data length: 4</td><td>Data: d0, d1, d2, d3</td></tr></table>			BG	Begin Motion			INS data length: 0	Data: n/a		ACK data length: 4	Data: d0, d1, d2, d3
	BG	Begin Motion										
		INS data length: 0	Data: n/a									
	ACK data length: 4	Data: d0, d1, d2, d3										
Example SET	⦿ <u>BG; Begin Motion</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 96 00 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 16 00 00 00 00 00 00 00 00 00 <i>Data [d3: d2: d1: d0] = 0x00000000; (don't care).</i>											
Note	– Activate the newly set parameters and initiate motion. This applies to JOG, PTP, and PVT motion control. The motor will remain stationary until BG is entered. Once BG is entered, motion parameters (e.g., SP/PR/PA/JV/PV) will be activated, and motion will begin. – For simultaneous movement of multiple UIM342 units, the user can first set parameters such as SP/PA for each node ID, and then send BG to the global or group ID.											

6.12 ST Stops Motion

Motion Control

CW	0x17 (No ACK); 0x97 (Need ACK)				
Data Format	n/a				
Description					
	ST	Stop Motion			
		INST data length: 0		Data: n/a	
		ACK data length: 0		Data: n/a	
Example SET					
	⦿ <u>ST; Stop Motion</u>				
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7				
	05 97 00 00 00 00 00 00 00 00 00				
ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7					
05 17 00 00 00 00 00 00 00 00 00					
Note	– Stop current movement, using stop deceleration (SD) to decelerate.				

6.13 MF Motion Parameter Frame

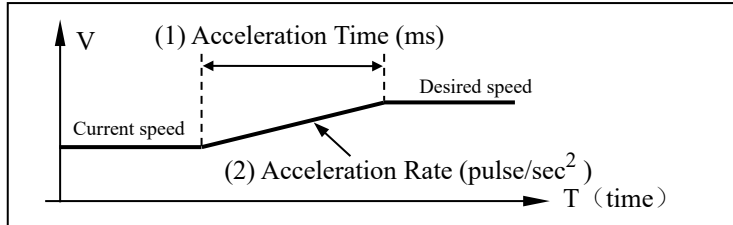
Motion Control

CW	0x18 (No ACK); 0x98 (Need ACK)																																																	
Data Format	unsigned 8-bit																																																	
Description	<table><tr><td>MF=N</td><td colspan="4">Set/select motion parameter group to be operated.</td></tr><tr><td></td><td>INST data length</td><td>1</td><td>Data</td><td>d0</td></tr><tr><td></td><td>ACK data length</td><td>1</td><td>Data</td><td>d0</td></tr></table>					MF=N	Set/select motion parameter group to be operated.					INST data length	1	Data	d0		ACK data length	1	Data	d0																														
	MF=N	Set/select motion parameter group to be operated.																																																
		INST data length	1	Data	d0																																													
		ACK data length	1	Data	d0																																													
	<table><tr><td>N</td><td colspan="4">Description</td></tr><tr><td>0</td><td colspan="4">Select the motion parameter group for normal operation</td></tr><tr><td>1</td><td colspan="4">Reserved</td></tr><tr><td>2</td><td colspan="4">Select the motion parameter group for the rising edge of IN1</td></tr><tr><td>3</td><td colspan="4">Select the motion parameter group for the falling edge of IN1</td></tr><tr><td>4</td><td colspan="4">Select the motion parameter group for the rising edge of IN2</td></tr><tr><td>5</td><td colspan="4">Select the motion parameter group for the falling edge of IN2</td></tr><tr><td>6</td><td colspan="4">Select the motion parameter group for the rising edge of IN3</td></tr><tr><td>7</td><td colspan="4">Select the motion parameter group for the falling edge of IN3</td></tr></table>					N	Description				0	Select the motion parameter group for normal operation				1	Reserved				2	Select the motion parameter group for the rising edge of IN1				3	Select the motion parameter group for the falling edge of IN1				4	Select the motion parameter group for the rising edge of IN2				5	Select the motion parameter group for the falling edge of IN2				6	Select the motion parameter group for the rising edge of IN3				7	Select the motion parameter group for the falling edge of IN3			
	N	Description																																																
	0	Select the motion parameter group for normal operation																																																
	1	Reserved																																																
	2	Select the motion parameter group for the rising edge of IN1																																																
	3	Select the motion parameter group for the falling edge of IN1																																																
4	Select the motion parameter group for the rising edge of IN2																																																	
5	Select the motion parameter group for the falling edge of IN2																																																	
6	Select the motion parameter group for the rising edge of IN3																																																	
7	Select the motion parameter group for the falling edge of IN3																																																	
Example SET	⊙ <u>MF=3; Select the motion parameter group for the falling edge of IN1</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 98 01 03 00 00 00 00 00 00 00 <i>Data [d0] = 0x03; (MF = 3).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 18 01 03 00 00 00 00 00 00 00																																																	
Note	– The UIM342 supports different motion parameter groups for preset actions triggered by different sources. MF=N provides a method to use same instruction (e.g. AC) to assign a motion parameter to different groups.																																																	
	Normal Operation <table><tr><td>AC [0]</td></tr><tr><td>DC [0]</td></tr><tr><td>SS [0]</td></tr><tr><td>SP [0]</td></tr><tr><td>PR [0]</td></tr><tr><td>PA [0]</td></tr></table> ↑ IN1 <table><tr><td>AC [2]</td></tr><tr><td>DC [2]</td></tr><tr><td>SS [2]</td></tr><tr><td>SP [2]</td></tr><tr><td>PR [2]</td></tr><tr><td>PA [2]</td></tr></table> ↓ IN1 <table><tr><td>AC [3]</td></tr><tr><td>DC [3]</td></tr><tr><td>SS [3]</td></tr><tr><td>SP [3]</td></tr><tr><td>PR [3]</td></tr><tr><td>PA [3]</td></tr></table> ↓ IN3 <table><tr><td>AC [7]</td></tr><tr><td>DC [7]</td></tr><tr><td>SS [7]</td></tr><tr><td>SP [7]</td></tr><tr><td>PR [7]</td></tr><tr><td>PA [7]</td></tr></table> – For example, to set the acceleration of IN1 falling edge to 1000, using: MF=1; AC=1000; To get the acceleration of IN1 falling edge, using: MF=1; AC; – After setting/getting a parameter, MF value will automatically reset to 0.					AC [0]	DC [0]	SS [0]	SP [0]	PR [0]	PA [0]	AC [2]	DC [2]	SS [2]	SP [2]	PR [2]	PA [2]	AC [3]	DC [3]	SS [3]	SP [3]	PR [3]	PA [3]	AC [7]	DC [7]	SS [7]	SP [7]	PR [7]	PA [7]																					
AC [0]																																																		
DC [0]																																																		
SS [0]																																																		
SP [0]																																																		
PR [0]																																																		
PA [0]																																																		
AC [2]																																																		
DC [2]																																																		
SS [2]																																																		
SP [2]																																																		
PR [2]																																																		
PA [2]																																																		
AC [3]																																																		
DC [3]																																																		
SS [3]																																																		
SP [3]																																																		
PR [3]																																																		
PA [3]																																																		
AC [7]																																																		
DC [7]																																																		
SS [7]																																																		
SP [7]																																																		
PR [7]																																																		
PA [7]																																																		

Motion Controller with CAN Interface

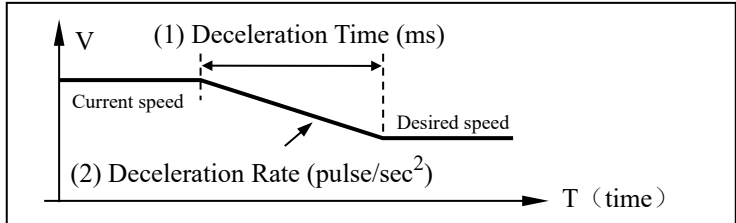
6.14 AC Acceleration

Motion Control

CW	0x19 (No ACK); 0x99 (Need ACK)									
Data Format	unsigned 32-bit; Range: 0...2 ³² pulse/sec ²									
Description	<table><tr><td>AC</td><td colspan="2">Get Acceleration</td></tr><tr><td>INS data length: 0</td><td>Data: n/a</td></tr><tr><td>ACK data length: 4</td><td>Data: d0, d1, d2, d3</td></tr></table>			AC	Get Acceleration		INS data length: 0	Data: n/a	ACK data length: 4	Data: d0, d1, d2, d3
	AC	Get Acceleration								
	INS data length: 0	Data: n/a								
	ACK data length: 4	Data: d0, d1, d2, d3								
	<table><tr><td>AC=N</td><td colspan="2">Set Acceleration</td></tr><tr><td>INS data length: 4</td><td>Data: d0, d1, d2, d3</td></tr><tr><td>ACK data length: 4</td><td>Data: d0, d1, d2, d3</td></tr></table>			AC=N	Set Acceleration		INS data length: 4	Data: d0, d1, d2, d3	ACK data length: 4	Data: d0, d1, d2, d3
AC=N	Set Acceleration									
INS data length: 4	Data: d0, d1, d2, d3									
ACK data length: 4	Data: d0, d1, d2, d3									
When IC[4]=0, acceleration is defined as rate, N=1...65,000,000 (pulse/sec ²). When IC[4]=1, acceleration is defined as time, N=1...60,000 (ms)										
										
Example GET	○ <u>AC; Get Acceleration.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 99 00 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 19 04 E8 03 00 00 00 00 00 00 <i>Data [d3: d2: d1: d0] = 0x000003E8 = 1000. (AC = 1000).</i>									
Example SET	⦿ <u>AC=500; Set Acceleration 500.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 99 04 F4 01 00 00 00 00 00 00 <i>Data [d3: d2: d1: d0] = 0x000001F4 = 500.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 19 04 F4 01 00 00 00 00 00 00									
Note	– The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off. – After setting up, AC will take effect when the next BG is commanded.									

6.15 DC Deceleration

Motion Control

CW	0x1A (No ACK); 0x9A (Need ACK)								
Data Format	Unsigned 32-bit; Range: 0...2 ³² pulse/sec ²								
Description	<table><tr><td>DC</td><td colspan="2">Get Deceleration</td></tr><tr><td>INS data length: 0</td><td>Data: n/a</td></tr><tr><td>ACK data length: 4</td><td>Data: d0, d1, d2, d3</td></tr></table>		DC	Get Deceleration		INS data length: 0	Data: n/a	ACK data length: 4	Data: d0, d1, d2, d3
	DC	Get Deceleration							
	INS data length: 0	Data: n/a							
	ACK data length: 4	Data: d0, d1, d2, d3							
	<table><tr><td>DC=N</td><td colspan="2">Set Deceleration</td></tr><tr><td>INS data length: 4</td><td>Data: d0, d1, d2, d3</td></tr><tr><td>ACK data length: 4</td><td>Data: d0, d1, d2, d3</td></tr></table>		DC=N	Set Deceleration		INS data length: 4	Data: d0, d1, d2, d3	ACK data length: 4	Data: d0, d1, d2, d3
DC=N	Set Deceleration								
INS data length: 4	Data: d0, d1, d2, d3								
ACK data length: 4	Data: d0, d1, d2, d3								
When IC[4]=0, deceleration is defined as rate, N=1...65,000,000 (pulse/sec ²). When IC[4]=1, deceleration is defined as time, N=1...60,000 (ms)									
									
Example GET	○ <u>DC; Get Deceleration.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9A 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 1A 04 E8 03 00 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0x000003e8 =1000. (The Deceleration is 1000)</i>								
Example SET	⦿ <u>DC=1000; Set Deceleration to 1000.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9A 04 E8 03 00 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0x000003e8 (SET Deceleration 1000).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 1A 04 E8 03 00 00 00 00 00								
Note	– The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off. – After setting up, DC will take effect when the next BG is commanded.								

Motion Controller with CAN Interface

6.16 SS Cut-in Speed

Motion Control

CW	0x1B (No ACK); 0x9B (Need ACK)		
Data Format	Unsigned 32-bit, Range: $0 \dots 2^{32}$ pulse/sec ²		
Description	SS	Get Cut-in speed	
		INS data length: 0	Data: n/a
		ACK data length: 4	Data: d0, d1, d2, d3
	SS=N	Set Cut-in speed	
		INS data length: 4	Data: d0, d1, d2, d3
		ACK data length: 4	Data: d0, d1, d2, d3
Example GET	○ <u>SS; Get Cut-in speed.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9B 00 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 1B 04 E8 03 00 00 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0x000003e8 = 1000. (Cut-in Speed is 1000).</i>		
Example SET	⊙ <u>SS=1001; Set Cut-in speed 1001.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9C 04 00 6A 18 00 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0x000003e9 = 1001. (SET cut-in speed 1001).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9C 04 00 6A 18 00 00 00 00 00		
Note	– The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off.		

6.17 SD Stop Deceleration

Motion Control

CW	0x1C (No ACK); 0x9C (Need ACK)																																	
Data Format	unsigned 32-bit, Range: 0...2 ³² pulse/sec ²																																	
Description	<table border="1"> <thead> <tr> <th>SD</th><th colspan="4">Get Stop Deceleration</th></tr> </thead> <tbody> <tr> <td></td><td>INST data length</td><td>0</td><td>Data</td><td>n/a</td></tr> <tr> <td></td><td>ACK data length</td><td>4</td><td>Data</td><td>d0, d1, d2, d3</td></tr> </tbody> </table> <table border="1"> <thead> <tr> <th>SD=N</th><th colspan="4">Set Stop Deceleration</th></tr> </thead> <tbody> <tr> <td></td><td>INST data length</td><td>4</td><td>Data</td><td>d0, d1, d2, d3</td></tr> <tr> <td></td><td>ACK data length</td><td>4</td><td>Data</td><td>d0, d1, d2, d3</td></tr> </tbody> </table>				SD	Get Stop Deceleration					INST data length	0	Data	n/a		ACK data length	4	Data	d0, d1, d2, d3	SD=N	Set Stop Deceleration					INST data length	4	Data	d0, d1, d2, d3		ACK data length	4	Data	d0, d1, d2, d3
SD	Get Stop Deceleration																																	
	INST data length	0	Data	n/a																														
	ACK data length	4	Data	d0, d1, d2, d3																														
SD=N	Set Stop Deceleration																																	
	INST data length	4	Data	d0, d1, d2, d3																														
	ACK data length	4	Data	d0, d1, d2, d3																														
Example GET	○ <u>SD; Get Stop Deceleration.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9C 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 1C 04 00 6A 18 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0x00186A00; (Stop Deceleration = 1,600,000).</i>																																	
Example SET	◎ <u>SD=1600000;Set Stop Deceleration 1,600,000.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9C 04 00 6A 18 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0xFFFFFC18; (Stop Deceleration = 1,600,000).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 1C 04 00 6A 18 00 00 00 00																																	
Note	– Mechanical movement should not be stopped abruptly, as it may cause impact and damage to the machine. Therefore, a suitable maximum acceleration value must be set, which is the purpose of the SD value. – The following situations will trigger the use of SD value: 1) Instruction ST. 2) The input logic of emergency stop is triggered. 3) Other situations need the emergency stop, such as soft bumper limit is hit. – The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off.																																	

Motion Controller with CAN Interface

6.18 JV Jog Velocity

Motion Control

CW	0x1D (No ACK); 0x9D (Need ACK)		
Data Format	signed 32-bit; range: $-2^{31} \dots +(2^{31} - 1)$ pulse/sec		
Description	JV	Get Current Speed	
		INST data length: 0	Data: n/a
		ACK data length: 4	Data: d0, d1, d2, d3
	JV=N	Set Desired Speed + Set Speed (Jog) Mode	
		INST data length: 4	Data: d0, d1, d2, d3
		ACK data length: 5	Data: d0, d1, d2, d3, d4
Example GET	○ <u>JV; Get current speed.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9D 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 1D 04 18 FC FF FF 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0xFFFFFC18 = -1000. (Current speed = -1000)</i>		
Example SET	⊙ <u>JV= -1000; Set desired velocity -1000.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9D 04 18 FC FF FF 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0xFFFFFC18; (JV = -1000)</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 2E 05 02 18 FC FF FF 00 00 00 00 <i>CW = 0x2E. (i.e., DV)</i> <i>Sub-index d0 = 2. (Desired Velocity)</i> <i>Data [d4:d3:d2:d1] = 0xFFFFFC18 = -1000. (Desired relative position -1000)</i>		
Note	– Instruction JV sets the desired motor speed and switches the control mode to speed mode. – JV will take effective after the following BG instruction. – Reply of the JV=N is the desired value DV[2] – Once set, if want to get the desired value of speed, DV[2] should be used.		

6.19 SP PTP Speed

Motion Control

CW	0x1E (No ACK); 0x9E (Need ACK)		
Data Format	signed 32-bit; range: $-2^{31} \dots +(2^{31} - 1)$ pulse/sec		
Description	SP	Get Current Speed	
		INS data length: 0	Data: n/a
		ACK data length: 4	Data: d0, d1, d2, d3
	SP=N	Set Desired Speed	
		INS data length: 4	Data: d0, d1, d2, d3
		ACK data length: 5	Data: d0, d1, d2, d3, d4
Example GET	○ <u>SP; Get current speed.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9E 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 1E 04 18 FC FF FF 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0xFFFFFC18 = -1000. (Current speed = -1000)</i>		
Example SET	⊙ <u>SP= +1000; Set desired velocity +1000.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9E 04 E8 03 00 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0x000003e8 = 1000.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 2E 05 02 E8 03 00 00 00 00 <i>CW = 0x2E. (i.e., DV, Desired Value)</i> <i>Sub-index d0 = 2. (Desired Speed)</i> <i>Data [d4:d3:d2:d1] = 0x000003e8 = 1000. (Desired speed = +1000)</i>		
Note	– In PTP mode, PR / PA should be set first, and then SP. – To set input logic speed, SP should be used instead of JV.		

Motion Controller with CAN Interface

6.20 PR Position Relative

Motion Control

CW	0x1F (No ACK); 0x9F (Need ACK)		
Data Format	signed 32-bit; range: $-2^{31} \dots (2^{31} - 1)$ pulse		
Description	PR	Get Current Relative Position	
		INS data length: 0	Data: n/a
		ACK data length: 4	Data: d0, d1, d2, d3
	PR=N	Set Desired Relative Position + Set Position (PTP) Mode	
		INS data length: 4	Data: d0, d1, d2, d3
		ACK data length: 5	Data: d0, d1, d2, d3, d4
Example GET	○ <u>PR; Get current relative position.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9F 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 1F 04 18 FC FF FF 00 00 00 00 <i>Data [d3: d2: d1: d0] = 0xFFFFFC18 = -1000. (Current relative position = -1000)</i>		
	◎ <u>PR= +1000; Set desired relative position +1000.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 9F 04 E8 03 00 00 00 00 00 <i>Data [d3: d2: d1: d0] = 0x000003e8 = +1000.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 2E 05 03 E8 03 00 00 00 00 <i>CW = 0x2E; (i.e., DV, Desired Value) Sub-index d0 = 3; (Desired Relative Position) Data [d4: d3: d2: d1] = 0x000003e8 = +1000. (Desired relative position is +1000)</i>		
Note	- Relative Position is the displacement added to the current position. - In PTP mode, PR / PA should be set first, and then SP. - PR will take effective after the following BG instruction.		

6.21 PA Position Absolute

Motion Control

CW	0x20 (No ACK); 0xA0 (Need ACK)	
Data Format	Signed 32-bit ; Range: $-2^{31} \dots +(2^{31} - 1)$ pulse	
Description	PA	Get Current Absolute Position
	INS data length: 0	Data: n/a
	ACK data length: 4	Data: d0, d1, d2, d3
	PA=N	Set Desired Absolute Position + Set Position (PTP) Mode
	INS data length: 4	Data: d0, d1, d2, d3
	ACK data length: 5	Data: d0, d1, d2, d3, d4
Example GET	○ <u>PA; Get current absolute position.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 A0 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 20 04 18 FC FF FF 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0xFFFFFC18 = -1000. (PA= -1000)</i>	
	⊙ <u>PA= 1000; Set desired absolute position +1000.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 A0 04 E8 03 00 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0x000003e8 = 1000.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 2E 05 04 E8 03 00 00 00 00 <i>CW = 0x2E. (i.e., DV, Desired Value)</i> <i>Sub-index [d0] = 4; (Desired Absolute Position)</i> <i>Data [d4:d3:d2:d1] = 0x000003e8 = 1000.</i> <i>(Desired absolute position +1000)</i>	
Note	– In PTP mode, PR / PA should be set first, and then SP. – PA will take effective after the following BG instruction.	

6.22 OG Set Origin

Motion Control

CW	0x21 (No ACK); 0xA1 (Need ACK)											
Data Format	n/a											
Description	<table><tr><td>OG</td><td colspan="2">Set the current position as origin, clear the position counter</td></tr><tr><td></td><td>INS data length: 0</td><td>Data: n/a</td></tr><tr><td></td><td>ACK data length: 0</td><td>Data: n/a</td></tr></table>			OG	Set the current position as origin, clear the position counter			INS data length: 0	Data: n/a		ACK data length: 0	Data: n/a
OG	Set the current position as origin, clear the position counter											
	INS data length: 0	Data: n/a										
	ACK data length: 0	Data: n/a										
Example SET	⦿ <u>OG; Set Origin.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 A1 00 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 21 00 00 00 00 00 00 00 00 00											
Note	– OG is disabled while the motor is moving. – The origin can be set via input logic during motion.											

6.23 BL Backlash Compensation

CW	0x2D (No ACK); 0xAD (Need ACK)		
Data Format	unsigned 32-bit; range: 0...65535		
Description	BL	Get Backlash Compensation Value	
		INS data length: 0	Data: n/a
		ACK data length: 4	Data: d0, d1, d2(=0), d3(=0)
	BL=N	Set Backlash Compensation Value	
		INS data length: 4	Data: d0, d1, d2(=0), d3(=0)
		ACK data length: 4	Data: d0, d1, d2(=0), d3(=0)
Example GET	○ <u>BL; Get Backlash Compensation Value.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 AD 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 2D 04 E8 03 00 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0x03E8 = 1000. (BL=1000 pulse)</i>		
Example SET	◎ <u>BL=1001; Set Backlash Compensation =1001.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 AD 00 E9 03 00 00 00 00 00 <i>Data [d3:d2:d1:d0] = 0x03E9 = 1001. (BL=1001 pulse)</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 2D 04 E9 03 00 00 00 00 00		
Note	– The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off.		

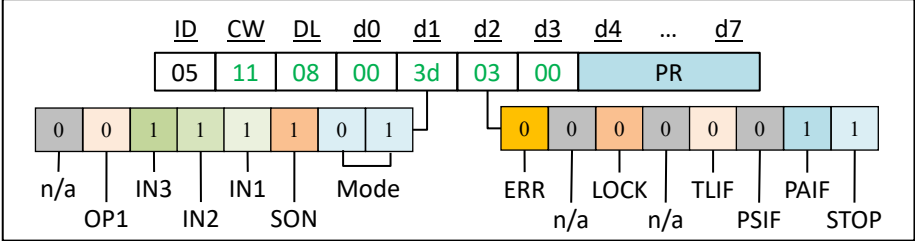
6.24 LM[i] Software Limits

Motion Control

CW	0x2C (No ACK); 0xAC (Need ACK)		
Data Format	i (unsigned 8-bit); Data (n/a)		
Description	LM[i]	Get Software Limits	
		INS data length: 1	Data: d0(=i)
		ACK data length: 5	Data: d0(=i), d1, d2, d3, d4
	LM[i]=N	Set Software Limits	
		INS data length: 5	Data: d0(=i), d1, d2, d3, d4
		ACK data length: 5	Data: d0(=i), d1, d2, d3, d4
	i	Description	Data [d4:d3:d2:d1]
	0	Maximum Speed	0...MCS*10000 pulse/sec
	1	Lower Working Limit	-2 ³¹ ...+2 ³¹ pulse
	2	Upper Working Limit	-2 ³¹ ...+2 ³¹ pulse
	3	Lower Bumping Limit	-2 ³¹ ...+2 ³¹ pulse
	4	Upper Bumping Limit	-2 ³¹ ...+2 ³¹ pulse
	6	Maximum Position Error (= QE[1])	50...65535 pulse
	7	Maximum Acceleration/Deceleration	1...1000000 (x1000 pulse/sec ²)
	254	Reset working / bumping limits	1 (reset only)
	255	Enable/disable software limits	0: disable; 1: enable
Note: LM[255] is not saved. Every time power up, LM[255] will be loaded from IC[7].			
Example GET	○ <u>LM[1]; Get Lower Working Limit.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 AC 01 01 00 00 00 00 00 00 <i>Index [d0] = 0x01. (i.e., Lower Working Limit)</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 2C 05 01 1C 10 00 00 00 00 <i>Data [d4:d3:d2:d1] = 0x0000101C = 4124.</i>		
	● <u>LM[2]=10000; Set Upper Working Limit to 10000 pulse.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 AC 05 02 10 27 00 00 00 00 <i>Index [d0] = 0x02. (i.e., Upper Working Limit).</i> <i>Data [d4:d3:d2:d1] = 0x00001027 = 10000.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 24 05 02 10 27 00 00 00 00		
Example SET	● <u>LM[255]=0; Disable Software Limits.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 AC 05 FF 00 00 00 00 00 00 <i>Index [d0] = 0x02. (i.e., Upper Working Limit).</i> <i>Data [d4:d3:d2:d1] = 0x00001027 = 10000.</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 24 05 FF 00 00 00 00 00 00		

6.25 MS[i] Motion Status

Motion Control

CW	0x11 (No ACK); 0x91 (Need ACK)		
Data Format	i (unsigned 8-bit); Data (unsigned 8-bit)		
Description	MS[1]	Get the Current Speed and Absolute Position	
		INS data length: 1	Data: d0(=1)
		ACK data length: 8	Data: d0(=1), d1. . .d7
	MS[0]	Get Status Flags and Relative Position	
		INS data length: 1	Data: d0(=0)
		ACK data length: 8	Data: d0(=0), d1. . .d7
	MS[0]=0	Clear Status Flags	
		INS data length: 2	Data: d0(=0), d1(=0)
		ACK data length: 8	Data: d0(=0), d1. . .d7
Please refer to chapter 3.7 “ Acquire Motion Status ” for details.			
Example GET	○ <u>MS[0]; Get Current Motion Status.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 91 01 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 11 08 00 3D 03 00 18 FC FF FF <i>Data [d0] = 0x00 (Index = 0).</i> <i>Data [d1] = 0x3D; Data [d2] = 0x03;</i>		
	 <i>Current Relative Position [d7:d6:d5:d4] = 0xFFFFFC18 = -1000 (pulse);</i>		
Example SET	● <u>MS[0]=0; Reset Motion Status.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 91 02 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 11 08 00 3D 01 00 18 FC FF FF		

Motion Controller with CAN Interface

6.26 DV[i] Desired Values

Motion Control

CW	0x2E (No ACK); 0xAE (Need ACK)		
Data Format	i (unsigned 8-bit); Data (n/a)		
Description	DV[i]	Get Desired Value	
		INS data length: 1	Data: d0(=i)
		ACK data length: 5	Data: d0(=i), d1, d2, d3, d4
	i	Description	Data [d4:d3:d2:d1]
	0	Current motion mode	0: JOG, 1: PTP
	1	Desired Working Current	0...80 : 0.0...8.0 Amp
	2	Desired Speed	-2 ³¹ ...+2 ³¹ pulse/sec
	3	Desired Relative Position	-2 ³¹ ...+2 ³¹ pulse
	4	Desired Absolute Position	-2 ³¹ ...+2 ³¹ pulse
Example GET	○ <u>DV[1]; Get Desired Working Current.</u>		
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7		
	05 AE 01 01 00 00 00 00 00 00 00		
	Index [d0] = 0x01. (i.e., Desired Working Current)		
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7		
	05 2E 05 01 1C 00 00 00 00 00 00		
	Data [d4:d3:d2:d1] = 0x000001C = 28.		
	(Desired Working Current = 2.8Amp)		
	○ <u>DV[2]; Get Desired Speed.</u>		
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7		
	05 AE 01 02 00 00 00 00 00 00 00		
	Index [d0] = 0x02. (i.e., Desired Speed)		
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7		
	05 2E 05 02 E8 03 00 00 00 00 00		
	Data [d4:d3:d2:d1] = 0x000003e8 = +1000.		
	(Desired Velocity = +1000 pulse/sec).		
	○ <u>DV[3]; Get Desired Relative Position.</u>		
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7		
	05 AE 01 03 00 00 00 00 00 00 00		
	Index [d0] = 0x03. (i.e., Desired Relative Position)		
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7		
	05 2E 05 03 30 F8 FF FF 00 00 00		
	Data [d4:d3:d2:d1] = 0xFFFFF830 = -2000.		
	(Desired Relative Position = -2000 pulse).		

6.27 IL[i] Input Logic

Input Logic

CW	0x34 (No ACK); 0xB4 (Need ACK)																			
Data Format	i (unsigned 8-bit); Data (unsigned 16-bit)																			
Description	<table><tr><td>IL[i]</td><td colspan="4">Get Sensor Trigger Action</td></tr><tr><td></td><td>INS data length</td><td>1</td><td>Data</td><td>d0 (=i)</td></tr><tr><td></td><td>ACK data length</td><td>3</td><td>Data</td><td>d0 (=i), d1, d2</td></tr></table>					IL[i]	Get Sensor Trigger Action					INS data length	1	Data	d0 (=i)		ACK data length	3	Data	d0 (=i), d1, d2
	IL[i]	Get Sensor Trigger Action																		
		INS data length	1	Data	d0 (=i)															
		ACK data length	3	Data	d0 (=i), d1, d2															
	<table><tr><td>IL[i]=X</td><td colspan="4">Set Sensor Trigger Action</td></tr><tr><td></td><td>INS data length</td><td>3</td><td>Data</td><td>d0 (=i), d1, d2</td></tr><tr><td></td><td>ACK data length</td><td>3</td><td>Data</td><td>d0 (=i), d1, d2</td></tr></table>					IL[i]=X	Set Sensor Trigger Action					INS data length	3	Data	d0 (=i), d1, d2		ACK data length	3	Data	d0 (=i), d1, d2
	IL[i]=X	Set Sensor Trigger Action																		
		INS data length	3	Data	d0 (=i), d1, d2															
	ACK data length	3	Data	d0 (=i), d1, d2																
<table><tr><td>i</td><td>Description</td></tr><tr><td>0</td><td>Action after IN1 is triggered</td></tr><tr><td>1</td><td>Action after IN2 is triggered</td></tr><tr><td>2</td><td>Action after IN3 is triggered</td></tr><tr><td>3</td><td>Behavior when motor stalls, N=0: lock down, N=1: freewheel</td></tr></table>					i	Description	0	Action after IN1 is triggered	1	Action after IN2 is triggered	2	Action after IN3 is triggered	3	Behavior when motor stalls, N=0: lock down, N=1: freewheel						
i	Description																			
0	Action after IN1 is triggered																			
1	Action after IN2 is triggered																			
2	Action after IN3 is triggered																			
3	Behavior when motor stalls, N=0: lock down, N=1: freewheel																			
Please refer to chapter 5.3 “ Configure Input Logic Action ” for details.																				
Example GET	○ <u>IL[0]; Get IN1 trigger action.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 B4 01 00 00 00 00 00 00 00 <i>Index [d0] = 0x00. (i.e., IN1).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 34 03 00 80 80 00 00 00 00 <i>Data [d1] = 0x80. (Falling edge: POT = disabled, Action Code = 0).</i> <i>Data [d2] = 0x80. (Rising edge: POT = disabled, Action Code = 0).</i>																			
	Example SET	⦿ <u>ILx 01 06 08; Set IN2 Trigger Action.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 B4 03 01 06 80 00 00 00 00 <i>Index [d0] = 0x01. (i.e., IN2).</i> <i>Data [d1] = 0x06. (Falling edge: POT = Enable; Action Code = 6).</i> <i>Data [d2] = 0x80. (Rising edge: POT = Disable; Action Code = 0).</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 34 03 01 06 80 00 00 00 00																		
Note		– The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off.																		

Motion Controller with CAN Interface

6.28 TG[i] Trigger

Input Logic

CW	0x35 (No ACK); 0xB5 (Need ACK)	
Data Format	i (unsigned 8-bit); Data (unsigned 16-bit)	
Description	TG[i]	Get The Input Trigger Mode
		INS data length: 1 Data: d0 (=i)
		ACK data length: 3 Data: d0 (=i), d1, d2
	TG[i]=N	Set The Input Trigger Mode
		INS data length: 3 Data: d0 (=i), d1, d2
		ACK data length: 3 Data: d0 (=i), d1, d2
	i	Description
	0	Trigger mode for IN1 (Port 1)
	1	Trigger mode for IN2 (Port 2)
	2	Trigger mode for IN3 (Port 3)
Example GET	N	Description
	0	Trigger continuously
	1 ... 60000	Trigger after Low pass filter, filter time 1...60000 ms
	Please refer to chapter 5.2 " Trigger Type and Input Filter " for details.	
Example SET	○ <u>TG[1]; Get IN2 Trigger Type.</u>	
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 B5 01 01 00 00 00 00 00 00 00 <i>Index [d0] = 0x01, (i.e., IN2).</i>	
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 35 03 01 00 00 00 00 00 00 00 <i>Data [d2:d1] = 0x0000. (TG[0] = 0, Continuous Type).</i>	
	◎ <u>TG[1]=100; Set IN1 Low-pass Filter Time Constant 100ms.</u>	
Note	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 B5 03 01 64 00 00 00 00 00 00 <i>Index [d0] = 0x01. (i.e., IN2).</i> <i>Data [d2:d1] = 0x0064 = 100. (Low-pass filter time 100ms).</i>	
	ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 35 03 01 64 00 00 00 00 00 00	
	– The set value will be saved to EEPROM only when MO=0; otherwise, it stays in RAM and is lost after power off.	

6.29 DI Digital I/O

Input Logic

CW	0x37 (No ACK); 0xB7 (Need ACK)																																																																																																				
Data Format	Segment Data																																																																																																				
Description	<table><tr><td>DI</td><td colspan="2">Get Digital Input & Output</td></tr><tr><td>INS data length: 0</td><td>Data: n/a</td></tr><tr><td>ACK data length: 4</td><td>Data: d0, d1, d2(=0), d3(=0)</td></tr></table>		DI	Get Digital Input & Output		INS data length: 0	Data: n/a	ACK data length: 4	Data: d0, d1, d2(=0), d3(=0)																																																																																												
	DI	Get Digital Input & Output																																																																																																			
	INS data length: 0	Data: n/a																																																																																																			
	ACK data length: 4	Data: d0, d1, d2(=0), d3(=0)																																																																																																			
	<table><tr><td>DIx d0...d7</td><td colspan="2">Set Digital Output</td></tr><tr><td>INS data length: 4</td><td>Data: d0, d1(=1), d2(=0), d3(=0)</td></tr><tr><td>ACK data length: 4</td><td>Data: d0, d1, d2(=0), d3(=0)</td></tr></table>		DIx d0...d7	Set Digital Output		INS data length: 4	Data: d0, d1(=1), d2(=0), d3(=0)	ACK data length: 4	Data: d0, d1, d2(=0), d3(=0)																																																																																												
DIx d0...d7	Set Digital Output																																																																																																				
INS data length: 4	Data: d0, d1(=1), d2(=0), d3(=0)																																																																																																				
ACK data length: 4	Data: d0, d1, d2(=0), d3(=0)																																																																																																				
Set Output Port																																																																																																					
<table><tr><td>ID</td><td>CW</td><td>DL</td><td>d0</td><td>d1</td><td>d2</td><td>d3</td><td>d4</td><td>...</td><td>d7</td></tr><tr><td>ID</td><td>B7</td><td>04</td><td>d0</td><td>d1</td><td>00</td><td>00</td><td colspan="3">00</td></tr><tr><td>bit7</td><td>bit6</td><td>bit5</td><td>bit4</td><td>bit3</td><td>bit2</td><td>bit1</td><td>bit0</td><td>bit7</td><td>bit6</td><td>bit5</td><td>bit4</td><td>bit3</td><td>bit2</td><td>bit1</td><td>bit0</td></tr><tr><td colspan="4"></td><td colspan="4">Output</td><td colspan="8"></td></tr><tr><td colspan="4"></td><td colspan="4">Mask bit = 0 : Ignor Output Setting</td><td colspan="8">Mask</td></tr><tr><td colspan="4"></td><td colspan="4">Mask bit = 1 : Set Output</td><td colspan="8"></td></tr></table>		ID	CW	DL	d0	d1	d2	d3	d4	...	d7	ID	B7	04	d0	d1	00	00	00			bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0					Output																Mask bit = 0 : Ignor Output Setting				Mask												Mask bit = 1 : Set Output																											
ID	CW	DL	d0	d1	d2	d3	d4	...	d7																																																																																												
ID	B7	04	d0	d1	00	00	00																																																																																														
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0																																																																																						
				Output																																																																																																	
				Mask bit = 0 : Ignor Output Setting				Mask																																																																																													
				Mask bit = 1 : Set Output																																																																																																	
Input & Output Status																																																																																																					
<table><tr><td>ID</td><td>CW</td><td>DL</td><td>d0</td><td>d1</td><td>d2</td><td>d3</td><td>d4</td><td>...</td><td>d7</td></tr><tr><td>ID</td><td>37</td><td>04</td><td>d0</td><td>d1</td><td>00</td><td>00</td><td colspan="3">00</td></tr><tr><td>bit7</td><td>bit6</td><td>bit5</td><td>bit4</td><td>bit3</td><td>bit2</td><td>bit1</td><td>bit0</td><td>bit7</td><td>bit6</td><td>bit5</td><td>bit4</td><td>bit3</td><td>bit2</td><td>bit1</td><td>bit0</td></tr><tr><td colspan="4"></td><td colspan="4">Input</td><td colspan="8"></td></tr><tr><td colspan="4"></td><td colspan="4">IN3</td><td colspan="4">IN2</td><td colspan="4">IN1</td></tr><tr><td colspan="4"></td><td colspan="4"></td><td colspan="8">Output</td></tr><tr><td colspan="4"></td><td colspan="4"></td><td colspan="8">OP1</td></tr></table>		ID	CW	DL	d0	d1	d2	d3	d4	...	d7	ID	37	04	d0	d1	00	00	00			bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0					Input																IN3				IN2				IN1												Output																OP1							
ID	CW	DL	d0	d1	d2	d3	d4	...	d7																																																																																												
ID	37	04	d0	d1	00	00	00																																																																																														
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0																																																																																						
				Input																																																																																																	
				IN3				IN2				IN1																																																																																									
								Output																																																																																													
								OP1																																																																																													
Example GET	○ <u>DI: Get digital input & output logic level.</u>																																																																																																				
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 B7 00 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 37 04 03 00 00 00 00 00 00 00 <i>d0.bit0 = 1. (IN1 = 1); d0.bit1 = 1. (IN2 = 1); d0.bit2 = 0. (IN3 = 0); d1.bit0 = 0. (OP1 = 0);</i>																																																																																																				
Example SET	◎ <u>DIx 01 01 00 00; Set output 1 logic level = HIGH.</u>																																																																																																				
	INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 B7 04 01 01 00 00 00 00 00 00 <i>d0.bit0 = 1. (OP1 = 1); d1.bit0 = 1. (OP1 Mask = 1);</i> ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 37 04 03 01 00 00 00 00 00 00 <i>d0.bit0 = 1. (IN1 = 1); d0.bit1 = 1. (IN2 = 1); d0.bit2 = 0. (IN3 = 0); d1.bit0 = 1. (OP1 = 1);</i>																																																																																																				

6.30 RT Real-Time Inform

Real-time Notification

CW	0x5A
Data Format	n/a
Description	Refer to chapter 2.4 “Real-Time Status and Alarm Notification” for details.
Example Message	○ <u>Real Time Information Message</u> ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 5A 01 01 10 00 00 00 00 00 00 <i>Data [d0] = 0x01 (Falling edge is detected on IN1).</i>

6.31 TI Temperature Indication

CW	0xBA (Need ACK)											
Data Format	Signed 16-bit											
Description	<table><tr><td>SN</td><td colspan="2">Get device's Serial Number</td></tr><tr><td></td><td>INS data length: 0</td><td>Data: n/a</td></tr><tr><td></td><td>ACK data length: 2</td><td>Data: d0...d1</td></tr></table>			SN	Get device's Serial Number			INS data length: 0	Data: n/a		ACK data length: 2	Data: d0...d1
	SN	Get device's Serial Number										
		INS data length: 0	Data: n/a									
	ACK data length: 2	Data: d0...d1										
Example GET	○ <u>TI; Get controller temperature.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 BA 00 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 3A 02 01 02 00 00 00 00 00 00 <i>[d1:d0] = 0x0201 (Temperature 53.1, i.e. 53.1 °C).</i> ○ <u>TI; Get controller temperature.</u> INS: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 BA 00 00 00 00 00 00 00 00 00 ACK: ID CW DL d0 d1 d2 d3 d4 d5 d6 d7 05 3A 02 F2 FF 00 00 00 00 00 00 <i>[d1:d0] = 0xFFFF2 (Temperature -14, i.e. -1.4 °C).</i>											

Appendix-1 RTU CRC16 Source Code

UIMessage uses CRC-16 (Modbus) algorithm. For details, please refer to Online CRC-8 CRC-16 CRC-32 Calculator (crccalc.com). The source code used in the calculation is listed below.

```
// CRC low byte lookup table (256 bytes)
unsigned char tblCRCLo[256] = {
0x00, 0xC0, 0xC1, 0x01, 0xC3, 0x03, 0x02, 0xC2, 0xC6, 0x06, 0x07, 0xC7, 0x05, 0xC5, 0xC4,
0x04, 0xCC, 0x0C, 0x0D, 0xCD, 0x0F, 0xCF, 0xCE, 0x0E, 0x0A, 0xCA, 0xCB, 0x0B, 0xC9, 0x09,
0x08, 0xC8, 0xD8, 0x18, 0x19, 0xD9, 0x1B, 0xDB, 0xDA, 0x1A, 0x1E, 0xDE, 0xDF, 0x1F, 0xDD,
0x1D, 0x1C, 0xDC, 0x14, 0xD4, 0xD5, 0x15, 0xD7, 0x17, 0x16, 0xD6, 0xD2, 0x12, 0x13, 0xD3,
0x11, 0xD1, 0xD0, 0x10, 0xF0, 0x30, 0x31, 0xF1, 0x33, 0xF3, 0xF2, 0x32, 0x36, 0xF6, 0xF7,
0x37, 0xF5, 0x35, 0x34, 0xF4, 0x3C, 0xFC, 0xFD, 0x3D, 0xFF, 0x3F, 0x3E, 0xFE, 0xFA, 0x3A,
0x3B, 0xFB, 0x39, 0xF9, 0xF8, 0x38, 0x28, 0xE8, 0xE9, 0x29, 0xEB, 0x2B, 0x2A, 0xEA, 0xEE,
0x2E, 0x2F, 0xEF, 0x2D, 0xED, 0xEC, 0x2C, 0xE4, 0x24, 0x25, 0xE5, 0x27, 0xE7, 0xE6, 0x26,
0x22, 0xE2, 0xE3, 0x23, 0xE1, 0x21, 0x20, 0xE0, 0xA0, 0x60, 0x61, 0xA1, 0x63, 0xA3, 0xA2,
0x62, 0x66, 0xA6, 0xA7, 0x67, 0xA5, 0x65, 0x64, 0xA4, 0x6C, 0xAC, 0xAD, 0x6D, 0xAF, 0x6F,
0x6E, 0xAE, 0xAA, 0x6A, 0x6B, 0xAB, 0x69, 0xA9, 0xA8, 0x68, 0x78, 0xB8, 0xB9, 0x79, 0xBB,
0x7B, 0x7A, 0xBA, 0xBE, 0x7E, 0x7F, 0xBF, 0x7D, 0xBD, 0xBC, 0x7C, 0xB4, 0x74, 0x75, 0xB5,
0x77, 0xB7, 0xB6, 0x76, 0x72, 0xB2, 0xB3, 0x73, 0xB1, 0x71, 0x70, 0xB0, 0x50, 0x90, 0x91,
0x51, 0x93, 0x93, 0x53, 0x52, 0x92, 0x96, 0x56, 0x57, 0x97, 0x55, 0x95, 0x94, 0x54, 0x9C, 0x5C,
0x5D, 0x9D, 0x5F, 0x9F, 0x9E, 0x5E, 0x5A, 0x9A, 0x9B, 0x5B, 0x99, 0x59, 0x58, 0x98, 0x88,
0x48, 0x49, 0x89, 0x4B, 0x8B, 0x8A, 0x4A, 0x4E, 0x8E, 0x8F, 0x4F, 0x8D, 0x4D, 0x4C, 0x8C,
0x44, 0x84, 0x85, 0x45, 0x87, 0x47, 0x46, 0x86, 0x82, 0x42, 0x43, 0x83, 0x41, 0x81, 0x80,
0x40 };

// CRC high byte lookup table (256 bytes)
unsigned char tblCRCHi[256] = {
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81,
0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01,
0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80,
0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01,
0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81,
0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01,
0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80,
0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01,
0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81,
0x40 };

// Calculate CRC, *buf points to "ID CW ... d8", qty = 12 bytes
unsigned short RtuCrc16(unsigned char* buf, unsigned int qty)
{
    unsigned char crcH = 0xFF;
    unsigned char crcL = 0xFF;
    int idx = 0;
    while (qty--)
    {
        {
            idx = crcL ^ *buf++;
            crcL = crcH ^ tblCRCHi[idx];
            crcH = tblCRCLo[idx];
        }
        return (crcH << 8) | crcL;
    }
}
```

Appendix-2 LED Indicator Patterns and Meanings

Major LED indicator patterns are listed below:

